

Crayon: Customized On-Device LLM via Instant Adapter Blending and Edge-Server Hybrid Inference

Jihwan Bang* Juntae Lee* Kyuhong Shim Seunghan Yang Simyung Chang[†]
Qualcomm AI Research[‡], Qualcomm Korea YH, Seoul, Republic of Korea
{jihwbang, juntlee, kshim, seunghan, simychan}@qti.qualcomm.com

Abstract

The customization of large language models (LLMs) for user-specified tasks gets important. However, maintaining all the customized LLMs on cloud servers incurs substantial memory and computational overheads, and uploading user data can also lead to privacy concerns. On-device LLMs can offer a promising solution by mitigating these issues. Yet, the performance of on-device LLMs is inherently constrained by the limitations of small-scaled models. To overcome these restrictions, we first propose Crayon, a novel approach for on-device LLM customization. Crayon begins by constructing a pool of diverse base adapters, and then we instantly blend them into a customized adapter without extra training. In addition, we develop a device-server hybrid inference strategy, which deftly allocates more demanding queries or non-customized tasks to a larger, more capable LLM on a server. This ensures optimal performance without sacrificing the benefits of on-device customization. We carefully craft a novel benchmark from multiple question-answer datasets, and show the efficacy of our method in the LLM customization.

1 Introduction

Large language model (LLM) has achieved unprecedented success on diverse natural language processing tasks such as machine translation, question and answering, text summarization and stylization, etc. Now then, it is expected for LLM-powered artificial intelligence (AI) to understand and satisfy each user’s unique needs such as recommendation systems, personalized assistance, and personalized search. To this end, a pivotal cornerstone is *customized LLM* where the LLM is highly advanced to a user-requested task. Indeed, several

web services related to LLM customization have been emerged such as GPTs (OpenAI, 2023) and PersonaAI (Character.AI, 2023; Meta, 2023).

However, due to the significant scale of LLM, keeping all the customized LLMs in the servers imposes a tremendous burden. Also, the privacy issues is inevitably raised by uploading the user’s data which entail user-requested task. Then, the focus is shifting towards *on-device LLM*. However, as the limited computing power of edge devices, it is impractical to address models as large as those on the servers. Therefore, for on-device LLM customization, it is crucial to maintain the performance on user-defined tasks, while constraining the model sizes. The practical method for the on-device LLM customization, however, has been less explored.

Recently, several works are developed to further lead out the ability of LLMs, and they may be exploited to cover the performance limit of LLMs on smaller size (device-level). Brown et al. (2020) introduced few-shot learning where a few example query-answer prompts are given together with users’ queries. Chain-of-thought (COT) (Wei et al., 2022) tried in-context learning by encouraging LLM to generate evidences as well as final answers. Also, for knowledge-intensive NLP tasks, retrieval augmented generation (RAG) (Lewis et al., 2020) made up query-relevant prompts by retrieving a given database. These prompt-based approaches have a intrinsic problem of increasing inference cost as the prompts get long and complex, and hence they are not suitable for edge devices.

Moreover, we can consider fine-tuning the on-device LLM in order to internalize these prompt-based knowledge for user-specific tasks to the model. Freezing the pre-trained LLM, adapter-based methods (Hu et al., 2021; Houlsby et al., 2019; Wang et al., 2022) have tried to facilitate LLM fine-tuning, and the low-rank-based adapter LoRA (Hu et al., 2021) have been most in the lime-light. Despite on-device-scale LLM armed with the

蜡笔：通过即时适配器混合和边缘服务器混合推理实现的定制化设备端大语言模型

方志焕* 李俊泰* 沈圭洪 杨承翰 张信荣[†] 高通人工智能研究[‡], 高通韩国 YH, 首尔, 大韩民国 {jihwbang, juntlee, kshim, seunghan, simychan}@qti.qualcomm.com

摘要

对大型语言模型（LLM）进行用户指定任务的定制变得越来越重要。然而，在云服务器上维护所有定制的 LLM 会产生大量的内存和计算开销，同时上传用户数据也可能引发隐私问题。设备端 LLM 可以通过缓解这些问题提供一种有前景的解决方案。然而，设备端 LLM 的性能本质上受到小规模模型限制的约束。为克服这些限制，我们首先提出了 Crayon，一种用于设备端 LLM 定制的新方法。Crayon 首先构建一个多样化的基础适配器池，然后我们无需额外训练即可即时将它们融合为一个定制适配器。此外，我们开发了一种设备 - 服务器混合推理策略，巧妙地将更高要求的查询或非定制任务分配给服务器上更大、更强的 LLM。这确保了在不牺牲设备端定制优势的情况下实现最佳性能。我们精心设计了一种新型

1 引言

大型语言模型（LLM）在多种自然语言处理任务上取得了前所未有的成功，例如机器翻译、问答、文本摘要和风格化等。如今，人们期望由 LLM 驱动的人工智能（AI）能够理解并满足每个用户的独特需求，例如推荐系统、个性化助手和个性化搜索。为此，一个关键的基石是定制化 LLM，即 LLM 高度针对用户请求的任务进行优化。事实上，已有若干

作者贡献相同。
[†] 表示通讯作者。
[‡] 高通人工智能研究是高通技术公司的一项计划。

与大型语言模型定制相关的网络服务已经出现，例如 GPTs (OpenAI, 2023) 和 PersonaAI (Character.AI, 2023; Meta, 2023).

然而，由于大语言模型（LLM）的规模非常庞大，将所有定制的 LLM 保存在服务器中会带来巨大的负担。此外，上传用户数据以执行用户请求的任务不可避免地会引发隐私问题。因此，关注点正逐渐转向设备端 LLM。然而，由于边缘设备的计算能力有限，处理与服务器上同样大的模型是不切实际的。因此，对于设备端 LLM 定制来说，在约束模型规模的同时保持用户定义任务的性能至关重要。然而，针对设备端 LLM 定制的实际方法仍然研究较少。

最近，已经开发了一些工作以进一步发挥大语言模型（LLM）的能力，并且它们可能被利用来弥补小规模（设备级）LLM 的性能限制。Brown 等人 (2020) 引入了少样本学习，其中提供少量示例查询 - 回答提示与用户的查询一起使用。思维链 (COT) (Wei 等人, 2022) 通过鼓励 LLM 生成证据以及最终答案来尝试上下文学习。此外，对于知识密集型的自然语言处理任务，检索增强生成 (RAG) (Lewis 等人, 2020) 通过检索给定数据库来生成与查询相关的提示。这些基于提示的方法存在一个内在问题，即随着提示变得越来越长和复杂，推理成本会增加，因此它们不适合边缘设备使用。

此外，我们可以考虑对设备端的 LLM 进行微调，以将这些基于提示的知识内化到模型中，以用于用户特定任务。冻结预训练的 LLM，基于适配器的方法（Hu 等人, 2021；Houlsby 等人, 2019；Wang 等人, 2022）已经尝试促进 LLM 的微调，而基于低秩的适配器 LoRA（Hu 等人, 2021）一直非常受关注。尽管设备端规模的 LLM 装备了

* The authors contribute equally.

[†] indicates corresponding author.

[‡]Qualcomm AI Research is an initiative of Qualcomm Technologies, Inc.

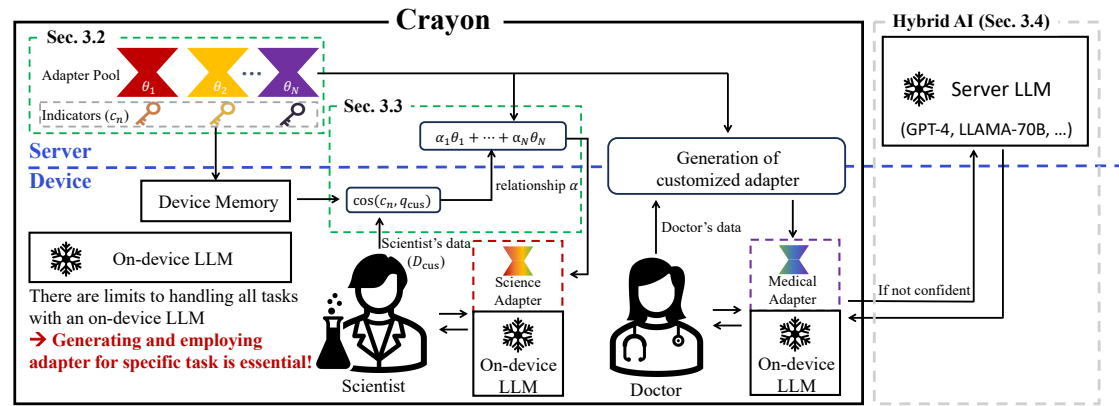


Figure 1: **Overall framework of the proposed method.** For on-device LLM customization without on-device training cost and privacy issue, we devise Crayon generating a suitable adapter instantly by utilizing an adapter pool including preparation of an adapter pool and deploying a customized adapter. Further, we also develop device-server hybrid inference to efficiently leverage a better generalized LLM in the server.

adapters, fine-tuning is time-consuming process and also needs a certain-level of training dataset. However, computing power of edge devices is limited and collecting enough user-specific data is also impractical. Thus, we raise inquiry: *How about simply customizing LLM without on-device training?*

For this purpose, given a target customization task, we propose Crayon customizing the on-device LLM via a single customized adapter which is blended on-the-fly from a set of base adapters, called *adapter pool*. To cover a wide range of user requests, the base adapters are learned to contain different knowledge each other. As depicted in Fig. 1, our approach requires no additional training cost in both of server and edge device when blending the customized adapter. In addition, we develop a device-server hybrid inference strategy to effectively leverage the better-generalized larger model of the server for handling unexpected queries (out-of-scope for the customized model). Our contributions are summarized as follows:

- We propose the first practical approach for customization of on-device small-scale LLM.
- We develop Crayon where the base adapters are learned satisfying their diversification by instantly blending the base adapters, and device-server hybrid inference to cover the out-of-customized tasks.
- We present an on-device LLM customization benchmark by tailoring the public question-answer datasets, and analyze our method.

2 Problem Set-up

Defining & processing customized task. In the context of few-shot learning (Brown et al., 2020), an LLM is prompted by several query-answer pairs to better understand testing queries. It has been proven that these few-shot prompts are helpful for increasing the generalization capability even in smaller LLMs. However, prompting increases the inference cost of LLMs as well, which is not preferred to on-device use case. Rather than prompting the few-shot examples $\mathcal{D}_c$, we define it as a target task specified to the user. From them, we immediately generate an adapter customized for the target task on the server, and deploy it to the on-device LLM. Note that, considering the privacy issue and communication cost with the server, $\mathcal{D}_c$ itself is never transmitted to the sever in our method.

Baseline LLM. For autoregressive, causal language model, most of popular LLMs such as GPT (Brown et al., 2020), LLaMA (Touvron et al., 2023a), Mistral (Jiang et al., 2023), and Falcon (Penedo et al., 2023) have adopted the decoder-only transformer (Vaswani et al., 2017) architecture. Hence, in our work, we select the smallest LLaMA (LLaMA-7B) as the baseline on-device LLM, which is reasonable size for edge devices¹.

Adapter for customized LLM. To reduce training cost, parameter efficient fine-tuning (PEFT) injects small trainable adapters, and only updates them for LLM fine-tuning. As a widely-used PEFT approach, LoRA (Hu et al., 2021) approximates the gradient of pre-trained weights into low-rank

¹<https://github.com/Bip-Rep/sherpa>

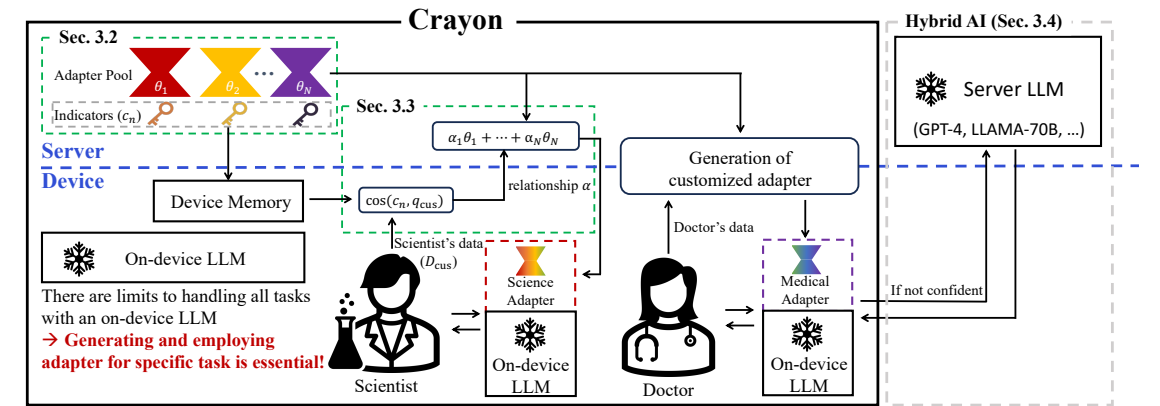


图 1: 所提出方法的总体框架。为了在不产生设备端训练成本和隐私问题的情况下进行设备端大语言模型 (LLM) 定制，我们设计了 Crayon，通过利用包含适配器池的适配器池创建合适的适配器，包括适配器池的准备和定制适配器的部署，从而即时生成适配器。此外，我们还开发了设备 - 服务器混合推理，以高效利用服务器端更具泛化能力的 LLM。

适配器，微调是一个耗时的过程，并且还需要一定级别的训练数据集。然而，边缘设备的计算能力有限，收集足够的用户特定数据也是不切实际的。因此，我们提出一个问题：是否可以在不进行设备端训练的情况下简单地定制大语言模型？

为此，针对特定的定制任务，我们提出 Crayon 通过一个单一的定制适配器来定制设备上的大型语言模型 (LLM)，该适配器是从一组基础适配器中即时混合而成，这组基础适配器称为适配器池。为了覆盖广泛的用户需求，基础适配器被学习以包含彼此不同的知识。如图 1 所示，当混合定制适配器时，我们的方法在服务器和边缘设备上都不需要额外的训练成本。此外，我们开发了一种设备 - 服务器混合推理策略，以有效利用服务器上更泛化的大模型来处理意外查询（对于定制模型而言超出其覆盖范围的查询）。我们的贡献总结如下：

- 我们提出了首个用于设备端小规模大语言模型定制的实用方法。
- 我们开发了 Crayon，其中基础适配器的学习过程通过即时混合基础适配器来满足它们的多样化，并采用设备 - 服务器混合推理来覆盖定制外任务。
- 我们通过定制公共问答数据集，提出了匿名设备 LLM 定制基准，并分析了我们的方法。

2 问题设定

定义与处理定制任务。在少样本学习的背景下 (Brown 等人, 2020)，大型语言模型 (LLM) 通过几个问答对来提示，从而更好地理解测试查询。已经证明，即便在较小的 LLM 中，这些少样本提示也有助于提高泛化能力。然而，提示也会增加 LLM 的推理成本，这在设备端使用的场景下是不理想的。与其提示少样本示例 $\mathcal{D}_c$ ，我们将其定义为用户指定的目标任务。从这些任务中，我们立即在服务器上生成针对目标任务的定制适配器，并将其部署到设备端的 LLM 中。需要注意的是，考虑到隐私问题和与服务器的通信成本， $\mathcal{D}_c$ 在我们的方法中，任务本身从未传输到服务器。

基线大型语言模型 (LLM)。对于自回归的因果语言模型，大多数流行的 LLM，例如 GPT (Brown 等., 2020)、LLaMA (Touvron 等., 2023a)、Mistral (Jiang 等., 2023) 以及 Falcon (Penedo 等., 2023) 都采用了仅解码器的 Transformer 架构 (Vaswani 等., 2017)。因此，在我们的工作中，我们选择最小的 LLaMA (LLaMA-7B) 作为基线的设备端 LLM，这对于边缘设备来说是合理的大小¹。

用于定制化大型语言模型的适配器。为了降低训练成本，参数高效微调 (PEFT) 引入了小型可训练适配器，并且在大型语言模型微调时仅更新它们。作为一种广泛使用的 PEFT 方法，LoRA (Hu 等., 2021) 将预训练权重的梯度近似为低秩

¹<https://github.com/Bip-Rep/sherpa>

matrices, and use them as the LLM adapters. As such, we also employ LoRA as our LLM adapter. Note that the learning LoRA does not take place on edge devices in our method. Briefly explaining our approach, we only train a set of N base LoRAs $\{l_{\theta_n}\}_{n=1}^N$ (i.e., LoRA pool) given a training set $\mathcal{D}_{\text{tr}}$ on the server, and then they are combined and deployed for instant LLM customization to the target task (i.e., $\mathcal{D}_{\text{c}}$) without additional training.

Device-server hybrid inference. Although an on-device LLM is well-customized to a user-specified task, there is inevitable performance gap between the device-level and server-level LLMs. Especially, the on-device LLM suffers from more performance drop when the inputs are out of the target task. Hence, we devise a device-server hybrid inference strategy. When output of an on-device LLM is unconfident, the output is replaced from the server’s larger model. To reduce frequent use of the server LLM, we develop a method to determine the reliability of on-device LLM’s output inside the device.

3 Methodology

In this section, we introduce Crayon which consists of LoRA pool construction (Sec. 3.2) and customized LoRA generation (Sec. 3.3). Also, we develop device-server consistent inference (Sec. 3.4).

3.1 Overall Framework of Crayon

As illustrated in Fig. 1, given $\mathcal{D}_{\text{tr}}$ that consists of various tasks and a baseline LLM $\mathcal{M}_{\Phi_0}$ where Φ_0 is the initial weight before customization, we jointly train N base LoRAs $\{l_{\theta_n}\}_{n=1}^N$ to have different characteristics and knowledge, respectively, in the server. Here, θ_n is the weight of l_{θ_n} . We also simultaneously learn the base LoRA indicator c_n which is allocated to l_{θ_n} . After training, N base LoRAs (i.e. LoRA pool) and the indicators are located in the server and device, respectively.

Then, for a small-scaled customization dataset $\mathcal{D}_{\text{c}}$, we first obtain the relationship between a LoRA pool and $\mathcal{D}_{\text{c}}$ by computing the similarities between the indicators and $\mathcal{D}_{\text{c}}$ on the device. This similarities are sent to the server, and then used to determine the weights of the base LoRAs in blending the customized LoRA. This customized LoRA is finally deployed to the user’s device to customize the on-device baseline LLM to the target task. Notice that we only upload the similarities of $\mathcal{D}_{\text{c}}$ to the indicators, but do not $\mathcal{D}_{\text{c}}$ itself. This is why our customization is privacy-friendly.

Algorithm 1: Learning LoRA pool	
Input:	baseline LLM $\mathcal{M}_{\Phi_0}$, base LoRAs $\{l_{\theta_n}\}_{n=1}^N$, training set $\mathcal{D}_{\text{tr}}$
1	# Extract intermediate embeddings
2	$Q_{\text{tr}} = \{q_x \mathcal{M}_{\Phi_0}^1(x), x \in \mathcal{D}_{\text{tr}}\}$
3	# Set the indicator of each base LoRA
4	$\{c_n\}_{n=1}^N = \text{K-MEANS_Centroids}(Q_{\text{tr}}, N)$
5	# Update the base LoRA weights
6	while not done do
7	Compute relationship $\{\alpha_n(q_x)\}_{n=1}^N$ (Eq. 2)
8	Compute the combined LoRA weight Θ_x (Eq. 3)
9	Update $\{\theta_n\}_{n=1}^N$ optimizing over Θ_x (Eq. 4)
10	end
Output: Weights of the base LoRAs $\theta_1, \dots, \theta_N$	

3.2 Crayon: LoRA Pool Construction

To address a variety of target customization tasks, it is important to diversify the base LoRAs’ knowledge and characteristics. To this end, we introduce an indicator for each base LoRA. In specific, for $\forall x \in \mathcal{D}_{\text{tr}}$, we first obtain the intermediate embeddings (empirically, the query embeddings of the first self-attention layer) as

$$q_x = \mathcal{M}_{\Phi_0}^1(x) \quad (1)$$

Then, we apply unsupervised k -means clustering ($k = N$) with q_x since the text corpora in $\mathcal{D}_{\text{tr}}$ have no specific task label. To suppress noise and focus on significant features, we reduce the dimensional of the embeddings using PCA before the clustering (see Appendix C for more details). For brevity, applying PCA is not explicitly notated.

N centroids c_n ’s resulting from k -means clustering are assigned to each base LoRA, dubbed base LoRA indicators. In the aftermentioned section, the base LoRAs are differently updated depending on the similarity between each corresponding indicator and embedding q_x during training.

Learning base LoRAs. For a training input x in $\mathcal{D}_{\text{tr}}$, we extract its query feature q_x as in the base LoRA indicators. Then, we compute its relationship with the base LoRA l_{θ_n} by using the corresponding indicator c_n :

$$\alpha_n(q_x) = \frac{\cos_sim(c_n, q_x) + 1}{2} \quad (2)$$

where $\cos_sim(\cdot, \cdot)$ denotes the cosine similarity.

To obtain a LoRA l_{Θ} specified to x , the N base LoRAs are combined by

$$\Theta_x = \alpha_1(q_x)\theta_1 + \alpha_2(q_x)\theta_2 + \dots + \alpha_N(q_x)\theta_N \quad (3)$$

Following (Hu et al., 2021), we only train the base LoRAs while freezing the baseline LLM. As

矩阵, 并将它们用作 LLM 适配器。因此, 我们也采用 LoRA 作为我们的 LLM 适配器。请注意, 在我们的方法中, LoRA 的学习并不在边缘设备上进行。简要说明我们的方法, 我们只在服务器上针对训练集训练一组 N 基础 LoRA $\{l_{\theta_n}\}_{n=1}^N$ (即 LoRA 池), 然后将它们组合并部署, 以便在无需额外训练的情况下即时定制 LLM 以适应目标任务 (即 $\mathcal{D}_{\text{c}}$)。

设备 - 服务器混合推理。虽然设备上的大语言模型 (LLM) 可以很好地针对用户指定的任务进行定制, 但设备级别与服务器级别的 LLM 之间不可避免地存在性能差距。尤其是, 当输入不属于目标任务时, 设备上的 LLM 性能下降更加明显。因此, 我们设计了一种设备 - 服务器混合推理策略。当设备上的 LLM 输出不自信时, 该输出将由服务器的更大模型替代。为了减少服务器 LLM 的频繁使用, 我们开发了一种在设备内部确定设备 LLM 输出可靠性的方法。

3 方法论

在本节中, 我们介绍 Crayon, 它包括 LoRA 池构建 (第 3.2 节) 和定制化 LoRA 生成 (第 3.3 节)。此外, 我们开发了设备 - 服务器一致推理 (第 3.4 节)。

3.1 Crayon 的整体框架

如图 1 所示, 给定 $\mathcal{D}_{\text{tr}}$ 由各种任务和一个基础大语言模型构成 $\mathcal{M}_{\Phi_0}$, 其中 Φ_0 是定制前的初始权重, 我们在服务器上联合训练 N 基础 LoRA $\{l_{\theta_n}\}_{n=1}^N$ 分别具有不同的特性和知识。在这里, θ_n 是 l_{θ_n} 的权重。我们还同时学习分配给 c_n 的基础 LoRA 指示器 l_{θ_n} 。训练后, N 基础 LoRA (即 LoRA 池) 和指标分别位于服务器和设备中。

然后, 对于一个小规模的定制数据集 $\mathcal{D}_{\text{c}}$, 我们首先通过在设备上计算指标与 $\mathcal{D}_{\text{c}}$ 之间的相似性, 获得 aLoRA 池与 $\mathcal{D}_{\text{c}}$ 之间的关系。然后将这些相似性发送到服务器, 并用于确定在混合定制 LoRA 时基础 LoRA 的权重。最终, 这个定制的 LoRA 被部署到用户的设备上, 以将设备上的基础 LLM 定制为目标任务。注意, 我们只上传 $\mathcal{D}_{\text{c}}$ 与指标的相似性, 但不会上传 $\mathcal{D}_{\text{c}}$ 本身。这就是我们的定制为什么是隐私友好的原因。

Algorithm 1: Learning LoRA pool

Input:	baseline LLM $\mathcal{M}_{\Phi_0}$, base LoRAs $\{l_{\theta_n}\}_{n=1}^N$, training set $\mathcal{D}_{\text{tr}}$
1	# Extract intermediate embeddings
2	$Q_{\text{tr}} = \{q_x \mathcal{M}_{\Phi_0}^1(x), x \in \mathcal{D}_{\text{tr}}\}$
3	# Set the indicator of each base LoRA
4	$\{c_n\}_{n=1}^N = \text{K-MEANS_Centroids}(Q_{\text{tr}}, N)$
5	# Update the base LoRA weights
6	while not done do
7	Compute relationship $\{\alpha_n(q_x)\}_{n=1}^N$ (Eq. 2)
8	Compute the combined LoRA weight Θ_x (Eq. 3)
9	Update $\{\theta_n\}_{n=1}^N$ optimizing over Θ_x (Eq. 4)
10	end
Output: Weights of the base LoRAs $\theta_1, \dots, \theta_N$	

3.2 蜡笔: LoRA 池构建

为了应对各种目标定制任务, 重要的是要使基础 LoRA 的知识和特性多样化。为此, 我们为每个基础 LoRA 引入了一个指标。具体来说, 对于 $\forall x \in \mathcal{D}_{\text{tr}}$, 我们首先获取中间嵌入 (经验上是第一层自注意力的查询嵌入) 作为

$$q_x = \mathcal{M}_{\Phi_0}^1(x) \quad (1)$$

然后, 我们应用无监督 k -均值聚类 ($k = N$) 与 q_x 因为 $\mathcal{D}_{\text{tr}}$ 中的文本语料没有特定的任务标签。为了抑制噪声并专注于重要特征, 我们在聚类之前使用 PCA 降低嵌入的维度 (参见附录 C 了解更多细节)。为了简洁起见, PCA 的应用未明确标注。

N 通过 k -均值聚类得到的质心 c_n 被分配给每个基础 LoRA, 称为基础 LoRA 指标。在后续章节中, 基础 LoRA 会根据训练过程中每个对应指标与嵌入 q_x 之间的相似性进行不同的更新。

学习基础 LoRAs. 对于训练输入 x 在 $\mathcal{D}_{\text{tr}}$ 中, 我们提取其查询特征 q_x 作为基础 LoRA 指标。然后, 我们使用相应的指标 c_n 计算它与基础 LoRA l_{θ_n} 的关系:

$$\alpha_n(q_x) = \frac{\cos_sim(c_n, q_x) + 1}{2} \quad (2)$$

其中 $\cos_sim(\cdot, \cdot)$ 表示余弦相似度。

要获得针对 x 指定的 LoRA l_{Θ} , N 基础 LoRA 通过

$$\Theta_x = \alpha_1(q_x)\theta_1 + \alpha_2(q_x)\theta_2 + \dots + \alpha_N(q_x)\theta_N \quad (3)$$

遵循 (胡等, 2021), 我们只训练基础 LoRA, 同时冻结基线 LLM。正如

Algorithm 2: Generate customized LoRA

Input: Base LoRAs $\{l_{\theta_n}\}_{n=1}^N$, indicators $\{c_n\}_{n=1}^N$, $\mathcal{M}_{\Phi_0}$, a few customized data $\mathcal{D}_c$

- 1 **On-device Processing**
- 2 # Get query embeddings from $\mathcal{D}_c$
- 3 $Q_c = \{q_{x,c} | \mathcal{M}_{\Phi_0}^1(x), x \in \mathcal{D}_c\}$
- 4 # Get user embedding
- 5 $q_c = \frac{1}{|Q_c|} \sum_{k=1}^{|Q_c|} q_{x,c}$, where $q_{x,c} \in Q_c$
- 6 # Compute relationship between q_c and $\{l_{\theta_n}\}_{n=1}^N$
- 7 $\alpha_n^c = \cos_sim(c_n, q_c), n = 1, \dots, N$
- 8 **Server Processing**
- 9 # Generate customized LoRA
- 10 $\hat{\theta} = \sum_{i=1}^N \alpha_i^c A_i B_i$ (Eq. 6)

Output: customized LoRA $l_{\hat{\theta}}$

such, we update the baseline LLM’s weights Φ_0 into $\Phi_0 + \Delta\Phi(\Theta_x)$ optimizing over Θ_x :

$$\max_{\Theta} \sum_{(x,y) \in \mathcal{D}_t} \sum_{t=1}^{|y|} \log(p_{\Phi_0 + \Delta\Phi(\Theta_x)}(y_t | x, y_{<t})) \quad (4)$$

where y is the label for x . The entire process is presented in [algorithm 1](#).

3.3 Crayon: Generation of Customized LoRA

We explain step-by-step in [algorithm 2](#). When a user provides a few examples $\mathcal{D}_c$ describing the customized task, we can generate the customized LoRA instantly from the LoRA pool. To do so, we first obtain an user embedding q_c to represent the customized task by averaging the query embeddings in $\mathcal{D}_c$ on the edge device:

$$q_c = \frac{1}{|\mathcal{D}_c|} \sum_{x \in \mathcal{D}_c} \mathcal{M}_{\Phi_0}^1(x) \quad (5)$$

After then, following [Eq. 2](#), we compute the combination ratios $\{\alpha_i^c\}_{i=1}^N$ based on the cosine similarities of q_c with the base LoRAs indicators $\{c_n\}_{n=1}^N$. We upload $\{\alpha_i^c\}_{i=1}^N$ to the server instead of $\mathcal{D}_c$ to generate the customized LoRA, and thus Crayon can protect the users’ privacy. On the server side, since the base LoRA’s weight θ_i can be decomposed by two low-rank trainable weights A_i and B_i , we instantly generate the customized LoRA $l_{\hat{\theta}}$:

$$\hat{\theta} = \sum_{i=1}^N \alpha_i^c A_i B_i \quad (6)$$

At last, the customized LoRA generated from the LoRA pool is deployed to the edge device, and it customizes the on-device LLM. Accordingly, we can effectively and efficiently customize the on-device LLM without additional training in both server and edge device.

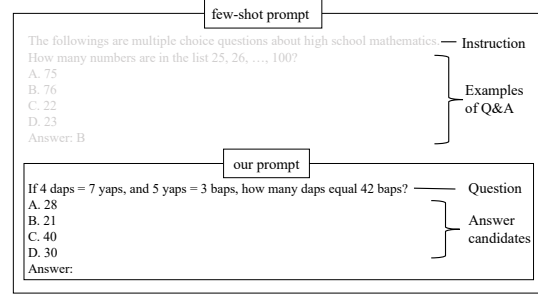


Figure 2: **Example prompt input in our method.** Different from few-shot prompt, this work does not utilize instruction and examples of QA.

3.4 Device-Server Hybrid Inference

Although the on-device LLM is customized, it cannot accommodate all kinds of input queries. For instance, the user can raise queries outside the scope of the customized task. Also, even for the queries raised from the customized task, the customized on-device LLM can suffer from its inherent limitation. i.e. relatively small model size. To overcome these difficulties, we intermittently turn to a larger LLM (e.g., LLaMA-30B) which is placed on the server due to high computational cost. This server LLM is not customized, but can make better response owing to its superb versatility.

Notice that when deciding if an input query x is routed to the server, we cannot utilize the server LLM $\mathcal{M}_{\Phi_s}$. Instead, we pre-compute a set of prototypes $S = \{s = \mathcal{M}_{\Phi_s}(x_c) | x_c \in \mathcal{D}_c\}$ where the server LLM’s characteristics is represented. We consider that S is deployed from the server to the edge device, together with $l_{\hat{\theta}}$.

Then, supposing that the desirable output of the on-device LLM may be close to S , we compare the on-device LLM’s output o_x with S . In specific, we compute the routing score r_x in the edge device:

$$r_x = \frac{1}{|S|} \sum_{s \in S} \cos_sim(o_x, s). \quad (7)$$

Finally, we route x to the server LLM when $r_x < r_{th}$, or hold the on-device output otherwise. The routing threshold r_{th} is empirically determined.

4 Experiment

As there is no established benchmark for on-device LLM customization, we present a novel benchmark for this field in [Sec. 4.1](#). Then, we show comprehensive evaluation and analyses in [Sec. 4.2](#) & [Sec. 4.3](#). Further details on experimental set-up and baselines in [Appendix A](#).

Algorithm 2: Generate customized LoRA

Input: Base LoRAs $\{l_{\theta_n}\}_{n=1}^N$, indicators $\{c_n\}_{n=1}^N$, $\mathcal{M}_{\Phi_0}$, a few customized data $\mathcal{D}_c$

- 1 **On-device Processing**
- 2 # Get query embeddings from $\mathcal{D}_c$
- 3 $Q_c = \{q_{x,c} | \mathcal{M}_{\Phi_0}^1(x), x \in \mathcal{D}_c\}$
- 4 # Get user embedding
- 5 $q_c = \frac{1}{|Q_c|} \sum_{k=1}^{|Q_c|} q_{x,c}$, where $q_{x,c} \in Q_c$
- 6 # Compute relationship between q_c and $\{l_{\theta_n}\}_{n=1}^N$
- 7 $\alpha_n^c = \cos_sim(c_n, q_c), n = 1, \dots, N$
- 8 **Server Processing**
- 9 # Generate customized LoRA
- 10 $\hat{\theta} = \sum_{i=1}^N \alpha_i^c A_i B_i$ (Eq. 6)

Output: customized LoRA $l_{\hat{\theta}}$

因此，我们将基线 LLM 的权重 Φ_0 更新到 $\Phi_0 + \Delta\Phi(\Theta_x)$ ，在 Θ_x 上进行优化：

$$\max_{\Theta} \sum_{(x,y) \in \mathcal{D}_t} \sum_{t=1}^{|y|} \log(p_{\Phi_0 + \Delta\Phi(\Theta_x)}(y_t | x, y_{<t})) \quad (4)$$

哪里 y 是 x 的标签。整个过程展示在[算法 1](#)中。

3.3 蜡笔：自定义 LoRA 的生成

我们在[算法 2](#)中逐步解释。当用户提供一些示例 $\mathcal{D}_c$ 描述自定义任务时，我们可以从 LoRA 池中即时生成自定义 LoRA。为此，我们首先获得一个用户嵌入 q_c 通过在边缘设备中对查询嵌入取平均来表示自定义任务： $\mathcal{D}_c$

$$q_c = \frac{1}{|\mathcal{D}_c|} \sum_{x \in \mathcal{D}_c} \mathcal{M}_{\Phi_0}^1(x) \quad (5)$$

然后，根据 [公式 2](#)，我们计算组合比率 $\{\alpha_i^c\}_{i=1}^N$ 基于 q_c 与基础 LoRA 指标的余弦相似度 $\{c_n\}_{n=1}^N$ 。我们上传 $\{\alpha_i^c\}_{i=1}^N$ 到服务器，而不是 $\mathcal{D}_c$ 来生成定制化 LoRA，因此 Crayon 可以保护用户的隐私。在服务器端，由于基础 LoRA 的权重 θ_i 可以通过两个低秩可训练权重 A_i 和 B_i 分解，我们即时生成定制化 LoRA $l_{\hat{\theta}}$:

$$\hat{\theta} = \sum_{i=1}^N \alpha_i^c A_i B_i \quad (6)$$

最后，从 LoRA 池生成的定制 LoRA 被部署到边缘设备上，并对设备上的 LLM 进行定制。因此，我们可以在服务器和边缘设备上无需额外训练的情况下，有效且高效地定制设备上的 LLM。

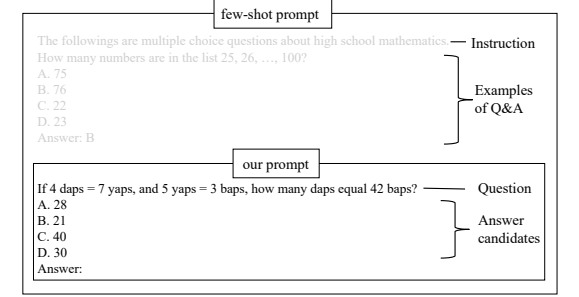


图 2：我们方法中的示例提示输入。与少量示例提示不同，本工作不使用问答的指令和示例。

3.4 设备 - 服务器混合推理

虽然设备上的大型语言模型（LLM）是定制的，但它无法处理所有类型的输入查询。例如，用户可能会提出超出定制任务范围的查询。此外，即使是来自定制任务的查询，定制的设备端 LLM 也可能受到自身固有限制的影响，即模型规模相对较小。为了解决这些困难，我们会间歇性地使用放置在服务器上的更大 LLM（例如，LLaMA-30B），因为其计算成本较高。该服务器 LLM 不是定制的，但由于其出色的通用性，能够给出更好的响应。

请注意，在决定输入查询 x 是否被路由到服务器时，我们不能使用服务器的 LLM $\mathcal{M}_{\Phi_s}$ 。相反，我们预先计算了一组原型

$S = \{s = \mathcal{M}_{\Phi_s}(x_c) | x_c \in \mathcal{D}_c\}$ ，其中表示了服务器 LLM 的特性。我们认为 S 是从服务器部署到边缘设备的，同时包含 $l_{\hat{\theta}}$ 。

然后，假设设备本地大型语言模型（LLM）的理想输出可能接近 S ，我们将设备本地 LLM 的输出 o_x 与 S 进行比较。具体而言，我们在边缘设备中计算路由得分 r_x ：

$$r_x = \frac{1}{|S|} \sum_{s \in S} \cos_sim(o_x, s). \quad (7)$$

最后，我们在 $r_x < r_{th}$ 时将 x 路由到服务器 LLM，否则保留设备上的输出。路由阈值 r_{th} 是通过经验确定的。

4 实验

由于尚未建立设备端大模型定制的基准，我们在[第 4.1 节](#)提出了该领域的新基准。然后，我们在[第 4.2 节](#)和[第 4.3 节](#)展示了全面的评估与分析。[实验设置和基准的更多细节](#)见附录 A。

Method	On-device LLM Size	STEM	Humanities	Social Sciences	Other	Average
LLaMA	7B	27.8	33.2	30.9	33.0	31.0
LLaMA (few-shot) [†]	7B	30.5	34.0	38.3	38.1	35.1
LLaMA	13B	35.0	43.5	45.9	42.6	41.1
LLaMA (few-shot) [†]	13B	35.8	45.0	53.8	53.3	46.9
Single LoRA	7B + 14M	33.2	44.6	43.4	44.6	40.7
Single LoRA (few-shot)	7B + 14M	29.7	33.1	33.1	39.2	33.5
LoraHub [‡]	7B + 14M	35.1	47.3	46.2	44.1	42.4
Crayon	7B + 14M	36.1	50.0	49.8	46.0	44.6
Crayon + Hybrid(20%)	7B + 14M	38.6	53.6	57.6	48.6	47.6

Table 1: **Acc (%) for MMLU tasks.** [†] indicates the reported performance from original paper (Touvron et al., 2023a) that utilize few-shot learning by following (Hendrycks et al., 2020). [‡] mostly follows (Huang et al., 2023), but we modify the base model (FLAN-T5 $\rightarrow$ LLaMA-7B) and upstream tasks (BBH $\rightarrow$ {SIQA, MCQA, OBQA}). For more details, see Appendix B.

4.1 On-device Customization Benchmark

Datasets. QA (question-answering) datasets are widely used in evaluation of LLM. Hence, to verify the effectiveness of Crayon, we select three public multiple choice QA datasets for training the pool of base LoRAs; Social Interaction QA (SIQA) (Sap et al., 2019) that focuses on the reasoning about people’s actions and their social implications, MedMCQA (MCQA) (Pal et al., 2022) that addresses real-world medical entrance exam questions, and Openbook QA (OBQA) (Mihaylov et al., 2018) that contains open book exams for assessing human understanding of a subject. For validating task generalization, we utilize MMLU (Press et al., 2022) that contains 57 subjects across STEM, the humanities, the social sciences, and more.

Customized task configuration. In customization datasets, it is expected to contain conversations for different users, where each conversation comprising a series of user’s question and LLM’s response. However, there are no publicly available datasets. Despite several dialog datasets such as shareGPT (Tey, 2022) and ChatAlpaca (Bian et al., 2023), they lack user identities for the dialogue and then it is difficult to measure the customization results in these datasets. In contrast, the MMLU dataset including individual question-answer data provides annotated by the subject categories. We therefore consider each subject as the interest specified to a user, i.e. a distinct customization task. In specific, we assume 57 distinct users with their own customization tasks. From this, we can quantitatively evaluate customization results in terms of accuracy (Acc). In this experiment, we set the size of customized dataset $|\mathcal{D}_c|$ as 10 for every user.

4.2 Main Results

Crayon attains customization. We evaluate the effectiveness of our method in comparison with several baselines: LLaMA (Touvron et al., 2023a), LoRA (Hu et al., 2021), and LoraHub (Huang et al., 2023). For LLaMA, we compare the proposed Crayon with the larger LLaMA-13B as well as the LLaMA-7B. For fair comparison with our approach, we report both zero and few-shot results. In zero-shot, the input is prompted as in our unified template Fig. 2. In few-shot, task-specific few-shot prompt including $\mathcal{D}_c$ is used following (Touvron et al., 2023a). In single LoRA, we train LoRA on top of the LLaMA-7B with $\mathcal{D}_{tr}$, and it is used universally all the customized tasks. Similar to our method, LoraHub combines pre-trained LoRAs to create a new one suitable for a new task. However, there are several limitations that assume specific upstream tasks with the corresponding datasets to individually pre-train all the LoRAs, and also requires a time-consuming searching process to determine the combination ratios of the pre-trained LoRAs (More details in Tab. 3).

As shown in Tab. 1, Crayon outperforms all the compared methods with only on-device LLM. Once the base LoRAs are established in the server, our method enables superior customization (40.7% vs 44.6%) compared to Single LoRA with $r = 4$ at an identical inference cost in the edge device. Interestingly, Crayon surpasses LoraHub (Huang et al., 2023) by 2.2% in average, although the base LoRAs are combined instantly. It is intuitive that the variety of base LoRAs is beneficial for the generalization ability of LoRA pool. Hence, the outperforming performance of the proposed Crayon indi-

Method	On-device LLM Size	STEM	Humanities	Social Sciences	Other	Average
LLaMA	7B	27.8	33.2	30.9	33.0	31.0
LLaMA (few-shot) [†]	7B	30.5	34.0	38.3	38.1	35.1
LLaMA	13B	35.0	43.5	45.9	42.6	41.1
LLaMA (few-shot) [†]	13B	35.8	45.0	53.8	53.3	46.9
Single LoRA	7B + 14M	33.2	44.6	43.4	44.6	40.7
Single LoRA (few-shot)	7B + 14M	29.7	33.1	33.1	39.2	33.5
LoraHub [‡]	7B + 14M	35.1	47.3	46.2	44.1	42.4
Crayon	7B + 14M	36.1	50.0	49.8	46.0	44.6
Crayon + Hybrid(20%)	7B + 14M	38.6	53.6	57.6	48.6	47.6

表 1: **MMLU 任务的准确率 (%)**。† 表示原论文报告的性能 (Touvron 等, 2023a), 该论文通过遵循 (Hendrycks 等, 2020) 的方法进行少样本学习。‡ 大致遵循 (Huang 等, 2023), 但我们修改了基础模型 (FLAN-T5 $\rightarrow$ LLaMA-7B) 和上游任务 (BBH $\rightarrow$ {SIQA, MCQA, OBQA})。更多详情见附录 B。

4.1 设备端自定义基准

数据集。 QA (问答) 数据集被广泛用于大型语言模型的评估。因此, 为了验证 Crayon 的有效性, 我们选择了三个公开的多项选择问答数据集来训练基础 LoRA 的池; 社会互动问答 (SIQA) (Sap 等人, 2019), 该数据集关注对人们行为及其社会影响的推理, MedMCQA (MCQA) (Pal 等人, 2022), 该数据集涉及现实世界的医学入学考试题目, 以及 Openbook QA (OBQA) (Mihaylov 等人, 2018), 该数据集包含用来评估人类对学科理解的开放书考试题目。为了验证任务的泛化能力, 我们使用了 MMLU (Press 等人, 2022), 该数据集包含涵盖 STEM、人文以及社会科学的 57 个学科

定制任务配置。 在定制数据集中, 预计包含不同用户的对话, 每个对话由一系列用户的问题和大语言模型 (LLM) 的回答组成。然而, 目前没有公开可用的数据集。尽管存在一些对话数据集, 例如 shareGPT (Tey, 2022) 和 ChatAlpaca (Bian 等, 2023), 它们缺乏对话中的用户身份, 因此很难在这些数据集中衡量定制结果。相比之下, MMLU 数据集包含按学科类别标注的个体问答数据。因此, 我们将每个学科视为指定给用户的兴趣, 即一个独特的定制任务。具体来说, 我们假设有 57 个具有自己定制任务的不同用户。基于此, 我们可以从准确率 (Acc) 的角度定量评估定制结果。在本实验中, 我们设定了尺寸

4.2 主要结果

Crayon 实现了定制化。 我们评估了我们方法的有效性, 并与几个基线方法进行了比较: LLaMA (Touvron 等, 2023a)、LoRA (Hu 等, 2021) 以及 LoraHub (Huang 等, 2023)。对于 LLaMA, 我们将提议的 Crayon 与更大的 LLaMA-13B 以及 LLaMA-7B 进行了比较。为了与我们的方法进行公平比较, 我们报告了零样本和少样本的结果。在零样本情况下, 输入按照我们的统一模板进行提示图 2。在少样本情况下, 使用了任务特定的少样本提示, 包括 $\mathcal{D}_c$, 遵循 (Touvron 等, 2023a)。在单 LoRA 中, 我们在 LLaMA-7B 上训练 LoRA, 使用 $\mathcal{D}_{tr}$, 并将其普遍用于所有定制任务。与我们的方法类似, LoraHub 结合了预训练的 LoRA 来创建

如表 1 所示, Crayon 在仅使用设备上的大语言模型 (LLM) 的情况下, 表现优于所有对比方法。一旦服务器上建立了基础 LoRA, 我们的方法能够实现比单一 LoRA 更优的定制化 (40.7% 对 44.6%), 并且在边缘设备上的推理成本相同。有趣的是, $r = 4$ Crayon 在平均性能上超过了 LoraHub (Huang 等, 2023) 2.2%, 尽管基础 LoRA 是即时组合的。直观来看, 基础 LoRA 的多样性有利于 LoRA 池的泛化能力。因此, 所提出的 Crayon 的优异性能表明

Task	Customized Data ($\mathcal{D}_c$)							
	STEM	Humanities	Social Science	Others				
	Elementary Mathematics	HS Physics	Jurisprudence	World Religion	HS Geography	Professional Psychology	Anatomy	Management
Elementary Mathematics	27.2	26.2	26.5	27.2	24.9	26.2	27.2	25.4
HS Physics	28.5	31.1	29.1	31.1	31.1	31.1	26.5	30.5
Jurisprudence	54.6	54.6	54.6	50.9	52.8	52.8	54.6	49.1
World Religion	69.0	70.2	70.2	70.2	69.0	67.8	69.0	69.0
HS Geography	58.1	56.1	57.1	59.1	60.1	55.6	59.1	59.1
Professional Psychology	40.8	41.2	41.5	40.5	38.2	41.8	41.7	36.6
Anatomy	46.7	49.6	49.6	47.4	44.4	43.7	49.6	42.2
Management	52.4	51.5	50.5	57.3	57.3	52.4	52.4	58.3

Table 2: **Ablation study for importance of customization data.** We randomly select two subjects in four categories (*i.e.*, STEM, humanities, social science, and others), and the customization performance (Acc) is consistently the highest when the model is customized by using the corresponding dataset.

cates that our joint training of all the base LoRAs at once produces their diversity, more effectively. This experiment will be addressed in Sec. 4.3.

Device-server hybrid inference. To assess the efficacy of device-server hybrid inference, we employ LLaMA-30B as the server LLM, which yields 53.2% Acc on MMLU in average. Moreover, we set the routing threshold r_{th} to satisfy 20% routing ratio, empirically. As shown in Tab. 1, with only 20% routing to sever model, the proposed hybrid inference ‘Crayon + hybrid 20%’ obtains 47.6% Acc, which is even better than fully using the large 13B model. Thus, our approach can boost the customized on-device LLM by efficiently intervening a more versatile server LLM.

4.3 Further Analyses

We extensively analyze the key components of the proposed method.

Impact of customized data. To assess the efficacy of customized data $\mathcal{D}_c$, we randomly select two different subjects (*i.e.*, two customization datasets) from each category in MMLU, and summarize the results in Tab. 2. This demonstrates that customized LoRAs, when generated from their corresponding subjects, perform better on their matched subjects than when they are created from unrelated subjects. For instance, on the Management subject, the LoRA generated from Management data can obtain 7.8% higher Acc than the LoRA generated from Jurisprudence data. We can see a similar trend in other subjects. Hence, in spite of a very small-scale customization data $\mathcal{D}_c$, it can contain significant generalization cue for the target customization task. Then, unlike the compared methods, our Crayon can effectively leverage it for LLM customization.

Diversity in LoRA pool. Fig. 3 plots that density distribution² of α for each task contained in the

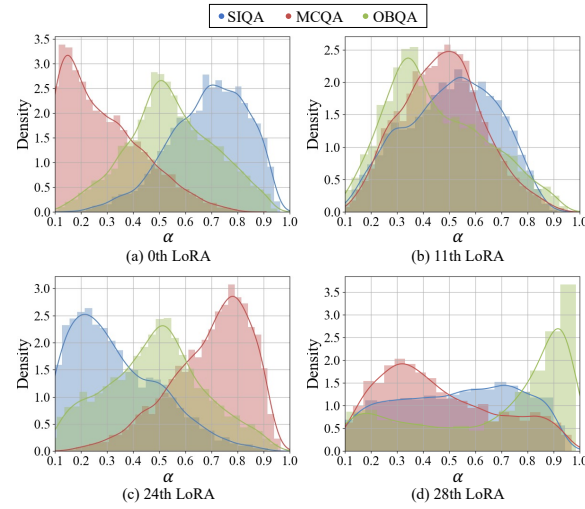


Figure 3: **Distribution plot of α for each training task on four base LoRAs.** In a, c, and d, the 0th, 24th and 28th base LoRAs have different preference on the SIQA, MCQA, and OBQA tasks, respectively. In b, the 11th base LoRA is trained on all the three tasks evenly.

training dataset, which shows the focus of the base LoRAs on the training tasks. From 32 base LoRAs, we select the four ones. We can identify that the trained base LoRAs have different weights for each task. 0th base LoRA (Fig. 3a), 24th base LoRA (Fig. 3c), and 28th base LoRA (Fig. 3d) more specialized to SIQA, MCQA, OBQA, respectively. Unlike above three base LoRAs, 11th base LoRA is likely to evenly trained with all the tasks as shown in Fig. 3b. Note that we do not provide any information to specify or define the task (e.g. task name) during training. However, Crayon produces a diversity of base LoRAs, enabling the LoRA pool to accommodate a wide range of customization tasks.

Device-server hybrid inference. In deep models, the confidence level is usually estimated by the maximum softmax score, and hence it can be a straightforward choice as decision rule for the hybrid inference. Hence, as in Fig. 4, we compare

Task	Customized Data ($\mathcal{D}_c$)							
	STEM	Humanities	Social Science	Others				
	Elementary Mathematics	HS Physics	Jurisprudence	World Religion	HS Geography	Professional Psychology	Anatomy	Management
Elementary Mathematics	27.2	26.2	26.5	27.2	24.9	26.2	27.2	25.4
HS Physics	28.5	31.1	29.1	31.1	31.1	31.1	26.5	30.5
Jurisprudence	54.6	54.6	54.6	50.9	52.8	52.8	54.6	49.1
World Religion	69.0	70.2	70.2	70.2	69.0	67.8	69.0	69.0
HS Geography	58.1	56.1	57.1	59.1	60.1	55.6	59.1	59.1
Professional Psychology	40.8	41.2	41.5	40.5	38.2	41.8	41.7	36.6
Anatomy	46.7	49.6	49.6	47.4	44.4	43.7	49.6	42.2
Management	52.4	51.5	50.5	57.3	57.3	52.4	52.4	58.3

表 2: 关于定制数据重要性的消融研究。我们在四个类别中随机选择了两个受试者（即 STEM、人文、社会科学和其他），并且当模型使用相应的数据集进行定制时，定制性能（Acc）始终最高。

表明我们一次性联合训练所有基础 LoRA 可以更有效地产生它们的多样性。本实验将在第 4.3 节中讨论。

设备 - 服务器混合推理。为了评估设备 - 服务器混合推理的有效性，我们使用 LLaMA-30B 作为服务器端大语言模型，在 MMLU 上平均取得 53.2% 的准确率。此外，我们设置了路由阈值 r_{th} 以经验性地满足 20% 的路由比例。如表 1 所示，仅有 20% 的请求路由到服务器模型，所提出的混合推理

“Crayon 混合 20%+”取得了 47.6% 的准确率，甚至优于完全使用大型 13B 模型。因此，我们的方法可以通过高效地介入更加通用的服务器大型语言模型来提升定制的设备端 LLM 的性能。

4.3 进一步分析

我们广泛分析了所提出方法的关键组成部分。

定制数据的影响。为评估定制数据 $\mathcal{D}_c$ 的 efficacy，我们在 MMLU 中随机从每个类别中选取两个不同的受试者（即两个自定义数据集），并在标签页 2 中总结结果。这表明，当自定义 LoRAs 从对应对象生成时，在匹配对象上的表现优于由无关子对象生成时。例如，在管理学主题中，由管理数据生成的 LoRA 比法学数据生成的 LoRA 获得 7.8% 的计取。我们在其他学科中也看到类似趋势。因此，尽管定制数据 $\mathcal{D}_c$ 规模非常小，但它可以包含目标定制任务的符号无法推广线索。因此，与对比方法不同，我们的蜡笔可以有效地利用它来实现 LLM c。

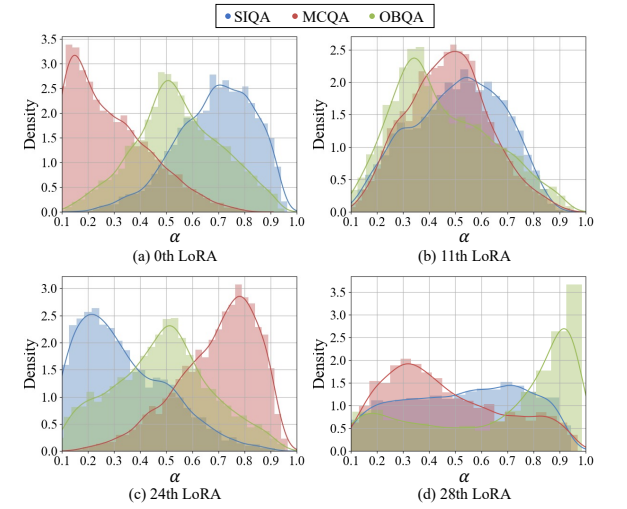


图 3: 每个训练任务在四个基础 LoRA 上的分布图。 α 在 a、c 和 d 中，第 0、第 24 和第 28 基础 LoRA 在 SIQA、MCQA 和 OBQA 任务上分别有不同的偏好。在 b 中，第 11 个基础 LoRA 在所有三个任务上均匀训练。

训练数据集，它显示了基础 LoRA 在训练任务中的关注点。从 32 个基础 LoRA 中，我们选择了四个。我们可以发现，训练好的基础 LoRA 对每个任务的权重不同。第 0 个基础 LoRA (图 3a)、第 24 个基础 LoRA (图 3c) 和第 28 个基础 LoRA (图 3d) 分别更专注于 SIQA、MCQA、OBQA。与上述三个基础 LoRA 不同，第 11 个基础 LoRA 可能在所有任务上均匀训练，如图 3b 所示。注意，在训练期间我们没有提供任何信息来指定或定义任务（例如任务名称）。然而，Crayon 生成了多样化的基础 LoRA，使 LoRA 池能够适应各种定制任务。

设备 - 服务器混合推理。在深度模型中，置信度通常通过最大 softmax 分数来估计，因此它可以作为混合推理的决策规则的直接选择。因此，如同图 4，我们进行比较

²It represents the proportion of the data in each range.

²它表示每个范围内数据的比例。

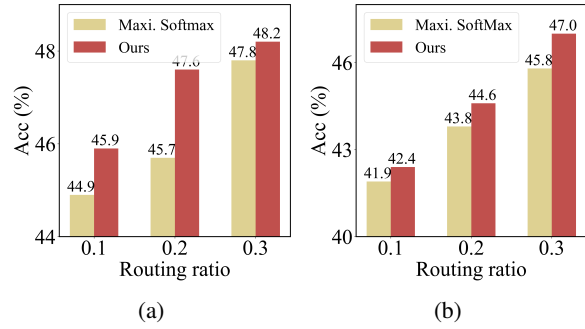


Figure 4: **Device-server hybrid inference varying routing ratio.** Acc (%) on (a) customized tasks and (b) mix of customized & out-of-customized tasks.

the proposed routing strategy with the maximum softmax-based approach varying the routing ratio. We can see the proposed approach beats the maximum softmax-based approach for all the three routing ratios in Fig. 4a. Hence, our method can effectively detect the failure cases of the customized on-device LLM under a routing ratio. To consider a more general setting, we also assess the capability to address the input queries outside the customized task. To this end, in Fig. 4b, the testing queries are configured from a non-customized subject as well as the customized one. In this setting, we can see a similar trend. Hence, the our routing effectively complements the customized on-device LLM, and completes the practical use of on-device LLM.

Hyperparameter sensitivity. Fig. 5 examines Crayon changing the size of the customization dataset ($|\mathcal{D}_c|$) and LoRA rank r . In Fig. 5a, when the number of base LoRAs N is set as 32 (default setting), the proposed method is capable of customization, irrespective of $|\mathcal{D}_c|$. This trend is mirrored when N is reduced to 16. Whereas, despite the same $|\mathcal{D}_c|$, the performance gap between $N = 16$ and $N = 32$ is notable. Hence, the number of base LoRAs highly impact the quality of customization. As the number of customized data will differ from user to user and N is usually predetermined in practical use, our method can be effectively applied in a real world scenario.

Further, we investigate the performance difference as varying rank of LoRA in Fig. 5b. When $r = 2$, the performance gap is marginal. We infer that the LoRA pool with too small rank might not effectively represent different customization tasks. Nevertheless, it is still slightly better than the a single universal LoRA without any additional training cost. However, once $r \geq 4$, the performance gap between them highly increases. Thus, for customiza-

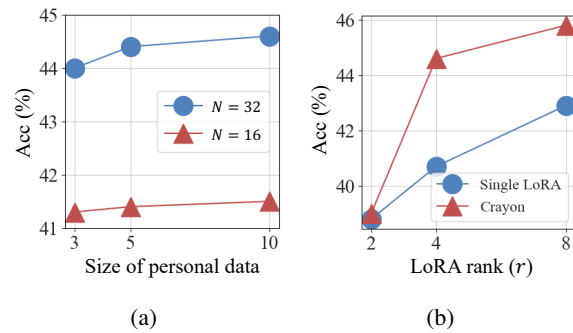


Figure 5: **Acc (%) according to (a) the size of the customized dataset and (b) LoRA rank.**

Training base LoRAs		Determining α		Acc (%)	Time (s)
Task-wise	Joint	Learning-based	Cos. sim.		
✓		✓		42.4	49.7
	✓	✓		44.5	51.0
	✓		✓	44.6	0.2

Table 3: **Ablation analysis.** Each component of Crayon is changed to the matched one of LoraHub. (1st and 3rd rows are LoraHub and complete Crayon, each.)

tion, the proposed task-wise base LoRA blending is more beneficial than generalizing a single LoRA.

Component ablation study. In Crayon, customization consists of two steps; i) constructing multiple base LoRAs and ii) deploying customized LoRA via blending the base LoRAs. First, we jointly learn the base LoRAs with no task definitions from the training datasets, which effectively diversifies the base LoRAs. Second, we obtain the relationship α via simply mapping to the LoRA indicators. LoraHub combines multiple task-specific LoRAs trained on different upstream tasks, and exhaustively search their relationship with a few examples $\mathcal{D}_c$. To validate the efficacy of those components, we change each component to the matched LoraHub’s one. As shown in Tab. 3, the first row where all the components are ablated corresponds to LoraHub. In the second row, we can infer that our task-agnostic joint learning is more beneficial to learn diverse base LoRAs, compared to the LoraHub’s individual LoRA learning with task-definition. Also, in the last row, rather than the time-consuming searching of the relationship α , the proposed simple LoRA indicator mapping is more proper to the jointly learned base LoRAs, since the base LoRAs are well-aligned with the LoRA indicators during the training.

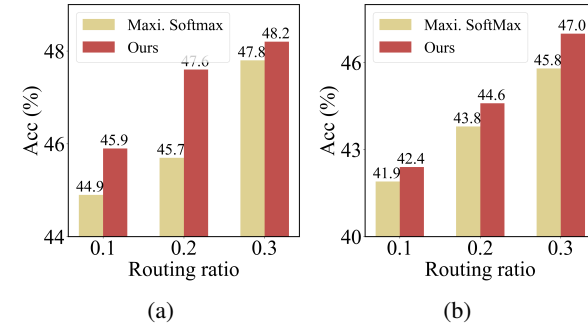


图 4: 设备 - 服务器混合推理随路由比例变化。在 (a) 定制任务和 (b) 定制任务与非定制任务混合上的准确率 (%)。

所提出的路由策略与基于最大 softmax 方法在不同路由比下的比较。可以看到，所提出的方法在图 Fig. 4a 中的三种路由比下均优于基于最大 softmax 的方法。因此，我们的方法可以有效检测定制化本地 LLM 在某一路路由比下的失败情况。为了考虑更一般的设置，我们还评估了应对定制任务之外的输入查询的能力。为此，在 Fig. 4b 中，测试查询既包括非定制化主题也包括定制化主题。在这种设置下，我们可以看到类似的趋势。因此，我们的路由策略有效地补充了定制化本地 LLM，并完成了本地 LLM 的实际应用。

超参数敏感性。 图 5 研究了 Crayon 改变自定义数据集的大小 ($|\mathcal{D}_c|$) 和 LoRA 秩 r 。在图 5a 中，当基础 LoRAs 的数量 N 设置为 32（默认设置）时，所提出的方法能够进行自定义，无论 $|\mathcal{D}_c|$ 如何。这个趋势在 N 减少到 16 时依然存在。然而，尽管具有相同的 $|\mathcal{D}_c|$ ， $N = 16$ 和 $N = 32$ 之间的性能差距仍然显著。因此，基础 LoRAs 的数量对自定义质量有很大影响。由于每个用户的自定义数据数量会不同，并且 N 在实际使用中通常是预先确定的，我们的方法可以有效地应用于实际场景。

此外，我们研究了在图 5b 中随 LoRA 秩变化的性能差异。当 $r = 2$ 时，性能差距很小。我们推测，秩过小的 LoRA 池可能无法有效地表示不同的定制任务。然而，它仍然略优于没有任何额外训练成本的单一通用 LoRA。然而，一旦 $r \geq 4$ ，它们之间的性能差距就会显著增加。因此，对于定制化

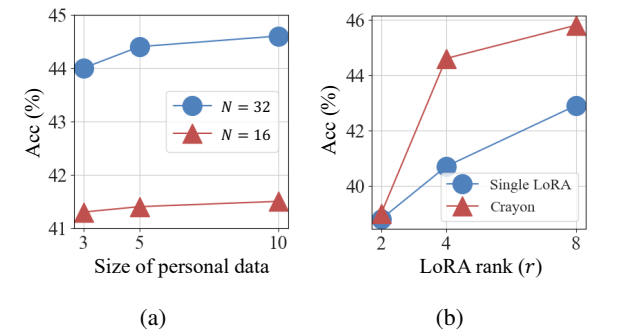


图 5: 根据 (a) 定制数据集的大小和 (b) LoRA 秩的准确率 (Acc %)。

Training base LoRAs		Determining α		Acc (%)	Time (s)
Task-wise	Joint	Learning-based	Cos. sim.		
✓		✓		42.4	49.7
	✓	✓		44.5	51.0
	✓		✓	44.6	0.2

表 3: **消融分析。** Crayon 的每个组件都被替换为 LoraHub 的对应组件。（第 1 行和第 3 行分别是 LoraHub 和完整的 Crayon。）

tion，所提出的按任务的基础 LoRA 混合比通用化单个 LoRA 更有益。

组件消融研究。 在 Crayon 中，定制化包括两个步骤；i) 构建多个基础 LoRA，ii) 通过混合基础 LoRA 部署定制化 LoRA。首先，我们在训练数据集上共同学习基础 LoRA，而不需任务定义，这有效地增加了基础 LoRA 的多样性。其次，我们通过简单映射到 LoRA 指标来获得关系 α 。LoraHub 结合了在不同上游任务上训练的多个特定任务 LoRA，并通过少量示例彻底搜索它们的关系 $\mathcal{D}_c$ 。为了验证这些组件的有效性，我们将每个组件替换为对应的 LoraHub 组件。如表 3 所示，第一行中所有组件都被消融的情况对应于 LoraHub。在第二行中，我们可以推断，相比于 LoraHub 的单独 LoRA 学习，我们的与任务无关的联合学习对于学习多样化基础 LoRA 更有益

5 Related Works

Task Generalization. In NLP, generalizing the language models to a wide range of unseen task has been important for its practical use. Addressing models with relatively small sizes (under 0.5B), several works have attempted to adapt them with few-shot examples into unseen tasks. However, CrossFit (Ye et al., 2021) has necessity of prompting the task names as hard prefixes. ReCross (Lin et al., 2022) alleviated this constraint via exploiting the retrieved training data, and yet requires an additional cost to retrieve task-friendly data.

Including a way more parameters, LLMs such as GPT (Brown et al., 2020), LLaMA (Touvron et al., 2023a,b), and Gemini (Team) has shown impressive results to a wide range of queries without specifically trained in the task of queries. However, their model sizes are too large for edge devices. LLaMA and Gemini also released their smaller versions under 10B training parameters which are feasible in edge devices, but their task generalization ability largely lags behind the larger models. To leverage these smaller LLMs, Mistral-7B (Jiang et al., 2023) applied sliding window attention and rolling buffer cache. Nevertheless, it cannot cover various tasks as much as the larger models. Recently, LoraHub (Huang et al., 2023) pre-trains LoRA adapters for multiple upstream tasks, and generate task-specific LoRA by mixing the pre-trained ones. However, it requires manipulating upstream tasks, and time-consuming process to determine mixing ratio of the pre-trained LoRAs. Contrarily, our method constructs the LoRA adapters which has different knowledge and characteristic each other, with no task ques.

Mixture of Experts (MOE). In MOE (Jacobs et al., 1991; Masoudnia and Ebrahimpour, 2014; Riquelme et al., 2021), each expert is controlled by a unique gating network, activated based on the distinct nature of the input data. Especially, in language domain, the MoE network identifies and engages the most suitable experts for every token. MoLoRA (Zadouri et al., 2023) and SiRA (Zhu et al., 2023) propose mixture of LoRA, and all the LoRAs and partial LoRAs (*i.e.*, top-K) are participating in the gating for every token, respectively. Moreover, very recently, Mixtral 8x7B (Jiang et al., 2024) has been introduced and surpasses LLaMA-2 70B across all evaluated benchmarks. It employs eight specialized experts that focus on dense matrices within fully connected layers. During the

processing of a token, a routing mechanism selects two of these experts, and their resulting outputs are then merged together. Since edge devices has limited storage to contain several LoRAs, token-wise MoE methods are hard to be applicable in our setup. Several MoE works such as Task-MoE (Kudugunta et al., 2021) and Skill Selection (Ponti et al., 2023) selects experts for every task, and can be adapted to on-device customization. However, they still assumed that the task id should be given with inputs while both training and inference phases.

Speculative Decoding. To accelerate LLM decoding, speculative decoding (Chen et al., 2023; Leviathan et al., 2023; Yang et al., 2023; Gupta et al., 2024) exploits a small (draft) model to predict what the larger target model will produce using uncertainty metric (Fadeeva et al., 2023), and then use the target model just to check if the prediction is correct. Device-server hybrid inference has a problem that the draft (*i.e.*, on-device) model cannot verify the prediction of its own for routing decision without target (*i.e.*, server) model. Even though the draft model’s output is correct, it should be verified after communication with the server. To solve it, AutoMix (Madaan et al., 2023) utilized meta-verifier to double-check the self-verification results. To improve both accuracy and efficiency, we develop advanced device-server hybrid inference where the on-device LLM’s reliability inside the device. Further, owing to the robust customization via Crayon, our customized on-device LLM do not need frequent intervention of the server LLM.

6 Conclusions

We propose Crayon and device-server hybrid inference for customizing on-device LLM for the first time. In Crayon, we jointly train a pool of base LoRAs, which has distinct knowledge and characteristics each other. Using this base LoRA pool, we can instantly blend the base LoRAs into a customized LoRA for user-defined task, without additional training or transferring user data to the server. To encompass complex queries, we develop the device-server hybrid system that measures the reliability of the customized LLM to identify the necessity of server LLM. We also present a new benchmark for quantitative evaluation of on-device LLM customization, incorporating commonly-used QA datasets, and our method shows superior performance. We hope that this benchmark can be a valuable tool for future research in this field.

5 相关工作

任务泛化。在自然语言处理（NLP）中，将语言模型泛化到各种未见过的任务对于其实际应用非常重要。针对相对较小的模型（低于 0.5B），一些研究尝试通过少量示例将它们适应于未见过的任务。然而，CrossFit（Ye 等, 2021）需要将任务名称作为硬前缀进行提示。ReCross（Lin 等, 2022）通过利用检索到的训练数据缓解了这一限制，但仍需要额外成本来检索适合任务的数据。

包括更多参数的 LLM，如 GPT（Brown 等人, 2020）、LLaMA（Touvron 等人, 2023a, b）和 Gemini（团队）已经在各种查询中显示出令人印象深刻的结果，而无需专门针对查询任务进行训练。然而，它们的模型规模对于边缘设备来说过大。LLaMA 和 Gemini 也发布了参数少于 10B 的较小版本，可以在边缘设备中运行，但它们的任务泛化能力在很大程度上落后于大模型。为了利用这些较小的 LLM，Mistral-7B（Jiang 等人, 2023）采用了滑动窗口注意力和滚动缓冲缓存。然而，它无法像大模型那样覆盖各种任务。最近，LoraHub（Huang 等人, 2023）为多个上游任务预训练了 LoRA 适配器，并

专家混合（MOE）。在 MOE 中（Jacobs 等人, 1991; Masoudnia 和 Ebrahimpour, 2014; Riquelme 等人, 2021），每个专家由一个唯一的门控网络控制，根据输入数据的不同特性激活。特别是在语言领域，MoE 网络能够识别并调用每个标记最合适的专家。MoLoRA（Zadouri 等人, 2023）和 SiRA（Zhu 等人, 2023）提出了 LoRA 的混合方案，所有 LoRA 和部分 LoRA（即，top-K）分别参与每个标记的门控。此外，最近推出的 Mixtral 8x7B（Jiang 等人, 2024）已经问世，并在所有评估基准中超越了 LLaMA-2 70B。它使用八个专门的专家

在处理一个令牌时，一个路由机制会选择其中两个专家，然后将它们产生的输出合并在一起。由于边缘设备的存储有限，无法容纳多个 LoRA，因此逐令牌的 MoE 方法在我们的设置中难以应用。几项 MoE 工作，如 Task-MoE（Kudugunta 等, 2021）和技能选择（Ponti 等, 2023）为每个任务选择专家，并且可以适应设备端的自定义。然而，他们仍然假设任务 ID 在训练和推理阶段都应与输入一起提供。**推测解码。**为了加速大型语言模型（LLM）的解码，推测解码（Chen 等, 2023；Leviathan 等, 2023；Yang 等, 2023；Gupta 等, 2024）利用一个小型（草稿）模型使用不确定性度量（Fadeeva 等, 2023）预测大型目标模型将生成的内容，然后仅使用目标模型来检查预测是否正确。设备 - 服务器混合推理存在一个问题，即草稿（即设备端）模型在没有目标（即服务器）模型的情况下无法验证其自身的预测以进行路由决策。即使草稿模型的输出是正确的，也应在与服务器通信后进行验证。为了解决这个问题，AutoMix（Madaan 等, 2023

6 结论

我们首次提出了 Crayon 与设备 - 服务器混合推理，用于定制化的设备端大语言模型（LLM）。在 Crayon 中，我们联合训练了一组基础 LoRA，每个 LoRA 都具有不同的知识和特性。利用这组基础 LoRA，我们可以即时将基础 LoRA 融合成针对用户定义任务的定制 LoRA，无需额外训练或将用户数据传输到服务器。为了应对复杂查询，我们开发了设备 - 服务器混合系统，该系统衡量定制 LLM 的可靠性，以确定是否需要服务器 LLM。我们还提出了一个用于量化评估设备端 LLM 定制的新基准，涵盖常用的问答数据集，并且我们的方法表现出优越的性能。我们希望这一基准能成为未来该领域研究的有价值工具。

7 Limitations

This work has demonstrated that through an adapter pool elaborately learned from a variety of datasets, it is possible to create customized adapters suitable for unseen target QA tasks. Our methodology is not limited to QA but is also anticipated to be applicable across more NLP domains that requires sequential output tokens. Alongside this, we expect that increasing the number of adapters in the adapter pool together with the utilization of more large-scale datasets will lead to the creation of more diversified customized adapters for a wider scope of unseen tasks.

8 Potential Risks

As with any LLM, the customized on-device LLM’s outputs might inadvertently perpetuate biases present in the training data, requiring careful oversight and potential intervention to ensure fairness and ethical use. Moreover, it is crucial to consider the battery life of the edge device of deploying additional computational resources, as the use of edge devices for intricate LLM tasks could result in increased energy consumption.

References

Ning Bian, Hongyu Lin, Yaojie Lu, Xianpei Han, Le Sun, and Ben He. 2023. Chatalpaca: A multi-turn dialogue corpus based on alpaca instructions. <https://github.com/cascip/ChatAlpaca>.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901.

Character.AI. 2023. <https://beta.character.ai/>.

Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. 2023. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*.

Ekaterina Fadeeva, Roman Vashurin, Akim Tsvigun, Artem Vazhentsev, Sergey Petrakov, Kirill Fedyanin, Daniil Vasilev, Elizaveta Goncharova, Alexander Panchenko, Maxim Panov, et al. 2023. Lm-polygraph: Uncertainty estimation for language models. *arXiv preprint arXiv:2311.07383*.

Neha Gupta, Harikrishna Narasimhan, Wittawat Jitkrit-tum, Ankit Singh Rawat, Aditya Krishna Menon, and Sanjiv Kumar. 2024. Language model cascades:

Token-level uncertainty and beyond. *arXiv preprint arXiv:2404.10136*.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2020. Measuring massive multitask language understanding. In *International Conference on Learning Representations*.

Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. 2019. Parameter-efficient transfer learning for nlp. In *International Conference on Machine Learning*, pages 2790–2799.

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models. *International Conference on Learning Representations*.

Chengsong Huang, Qian Liu, Bill Yuchen Lin, Tianyu Pang, Chao Du, and Min Lin. 2023. Lorahub: Efficient cross-task generalization via dynamic lora composition. *arXiv preprint arXiv:2307.13269*.

Robert A Jacobs, Michael I Jordan, Steven J Nowlan, and Geoffrey E Hinton. 1991. Adaptive mixtures of local experts. *Neural computation*, 3(1):79–87.

Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. 2023. Mistral 7b. *arXiv preprint arXiv:2310.06825*.

Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. 2024. Mixtral of experts. *arXiv preprint arXiv:2401.04088*.

Sneha Kudugunta, Yanping Huang, Ankur Bapna, Maxim Krikun, Dmitry Lepikhin, Minh-Thang Luong, and Orhan Firat. 2021. Beyond distillation: Task-level mixture-of-experts for efficient inference. *arXiv preprint arXiv:2110.03742*.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, pages 19274–19286. PMLR.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474.

Bill Yuchen Lin, Kangmin Tan, Chris Miller, Beiwen Tian, and Xiang Ren. 2022. Unsupervised cross-task generalization via retrieval augmentation. *Advances in Neural Information Processing Systems*, 35:22003–22017.

7 个限制

这项工作表明，通过从各种数据集中精心学习的适配器池，有可能创建适用于未知目标问答任务的定制适配器。我们的方法不仅限于问答任务，还预计可以应用于更多需要顺序输出标记的自然语言处理领域。与此同时，我们预计，通过增加适配器池中的适配器数量以及使用更多大规模数据集，将能够为更广泛的未知任务创建更多多样化的定制适配器。

8 个潜在风险

与任何大型语言模型一样，定制的本地设备大型语言模型的输出可能无意中延续训练数据中存在的偏见，因此需要仔细监督和潜在干预，以确保公平和道德使用。此外，考虑在边缘设备上增加额外计算资源时的电池续航也至关重要，因为在边缘设备上执行复杂的大型语言模型任务可能导致能源消耗增加。

参考文献

宁变, 林鸿宇, 卢耀杰, 韩先培, 孙乐, 和 何奔.

2023. Chatalpaca: 一个基于 alpaca 指令的多轮对话语料库 .<https://github.com/cascip/ChatAlpaca>.

汤姆·布朗、本杰明·曼、尼克·赖德、梅兰妮·苏比亚、贾里德·D·卡普兰、普拉富拉·达里瓦尔、阿尔文德·尼拉坎坦、普拉纳夫·沙姆、吉里什·萨斯特里、阿曼达·阿斯科尔 等人 . 2020. 语言模型是少量学习者 . 神经信息处理系统进展 , 33:1877–1901.

Character.AI. 2023. <https://beta.character.ai/>.

Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre 和 John Jumper. 2023. 使用推测采样加速大型语言模型解码。arXiv 预印本 arXiv:2302.01318。

Ekaterina Fadeeva, Roman Vashurin, Akim Tsvigun, Artem Vazhentsev, Sergey Petrakov, Kirill Fedyanin, Daniil Vasilev, Elizaveta Goncharova, Alexander Panchenko, Maxim Panov 等 . 2023. Lm- polygraph: 语言模型的不确定性估计。arXiv 预印本 arXiv:2311.07383.

Neha Gupta, Harikrishna Narasimhan, Wittawat Jitkrit-tum, Ankit Singh Rawat, Aditya Krishna Menon, 和 Sanjiv Kumar. 2024. 语言模型级联:

基于令牌的不确定性及其扩展。arXiv 预印本 arXiv:2404.10136。

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, 和 Jacob Steinhardt. 2020. 测量大规模多任务语言理解 . 载于 国际学习表征会议 .

Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan 和 Sylvain Gelly. 2019. 用于自然语言处理的参数高效迁移学习。在 国际机器学习会议 , 第 2790–2799 页。

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang 和 Weizhu Chen. 2021. Lora: 大语言模型的低秩适配。国际学习表征会议。

Chengsong Huang, Qian Liu, Bill Yuchen Lin, Tianyu Pang, Chao Du, 和 Min Lin. 2023. Lorahub: 通过动态 lora 组合实现跨任务高效泛化。arXiv 预印本 arXiv:2307.13269。

罗伯特·A·雅各布斯, 迈克尔·I·乔丹, 史蒂文·J·诺兰, 和杰弗里·E·辛顿。1991 年。《局部专家的自适应混合》。神经计算 , 3(1): 79–87。

Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier 等人 . 2023. Mistral 7b. arXiv 预印本 arXiv: 2310.06825.

Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand 等 . 2024. Mixtralofexperts. arXiv 预印本 arXiv:2401.04088.

Sneha Kudugunta、Yanping Huang、Ankur Bapna、Maxim Krikun、Dmitry Lepikhin、Minh-Thang Luong 和 Orhan Firat. 2021. 超越蒸馏：用于高效推理的任务级专家混合。arXiv 预印本 arXiv:2110.03742。

Yaniv Leviathan, Matan Kalman, 和 Yossi Matias. 2023. 通过推测解码进行变换器的快速推理。发表于 国际机器学习会议 , 页码 19274–19286. PMLR.

Patrick Lewis、Ethan Perez、Aleksandra Piktus、Fabio Petroni、Vladimir Karpukhin、Naman Goyal、Heinrich Küttler、Mike Lewis、Wen-tau Yih、Tim Rocktäschel 等人 . 2020. 用于知识密集型 NLP 任务的检索增强生成 . 神经信息处理系统进展 , 33:9459–9474.

林钰宸, 谭康民, 克里斯·米勒, 田北文, 任翔。2022. 通过检索增强实现无监督跨任务泛化。神经信息处理系统进展 , 35:22003–22017。

Aman Madaan, Pranjal Aggarwal, Ankit Anand, Srividya Pranavi Potharaju, Swaroop Mishra, Pei Zhou, Aditya Gupta, Dheeraj Rajagopal, Karthik Kappaganthu, Yiming Yang, et al. 2023. Automix: Automatically mixing language models. *arXiv preprint arXiv:2310.12963*.

Sourab Mangrulkar, Sylvain Gugger, Lysandre Debut, Younes Belkada, Sayak Paul, and Benjamin Bossan. 2022. Peft: State-of-the-art parameter-efficient fine-tuning methods. <https://github.com/huggingface/peft>.

Saeed Masoudnia and Reza Ebrahimpour. 2014. Mixture of experts: a literature survey. *Artificial Intelligence Review*, 42:275–293.

Meta. 2023. <https://ai.meta.com/genai/>.

Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. 2018. Can a suit of armor conduct electricity? a new dataset for open book question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2381–2391.

OpenAI. 2023. <https://openai.com/blog/introducing-gpts>.

Ankit Pal, Logesh Kumar Umapathi, and Malaikannan Sankarasubbu. 2022. Medmcqa: A large-scale multi-subject multi-choice dataset for medical domain question answering. In *Conference on Health, Inference, and Learning*, pages 248–260. PMLR.

Guilherme Penedo, Quentin Malartic, Daniel Hesslow, Ruxandra Cojocaru, Alessandro Cappelli, Hamza Alobeidli, Baptiste Pannier, Ebtesam Almazrouei, and Julien Launay. 2023. The refinedweb dataset for falcon llm: outperforming curated corpora with web data, and web data only. *arXiv preprint arXiv:2306.01116*.

Edoardo Maria Ponti, Alessandro Sordoni, Yoshua Bengio, and Siva Reddy. 2023. Combining parameter-efficient modules for task-level generalisation. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*, pages 687–702.

Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A Smith, and Mike Lewis. 2022. Measuring and narrowing the compositionality gap in language models. *arXiv preprint arXiv:2210.03350*.

Carlos Riquelme, Joan Puigcerver, Basil Mustafa, Maxim Neumann, Rodolphe Jenatton, André Susano Pinto, Daniel Keysers, and Neil Houlsby. 2021. Scaling vision with sparse mixture of experts. *Advances in Neural Information Processing Systems*, 34:8583–8595.

Maarten Sap, Hannah Rashkin, Derek Chen, Ronan Le Bras, and Yejin Choi. 2019. Social iqa: Commonsense reasoning about social interactions. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, pages 4463–4473.

Gemini Team. Gemini: A family of highly capable multimodal models. Technical report, Technical report, Google, 12 2023. URL <https://storage.googleapis.com>

Steven Tey. 2022. <https://sharegpt.com>.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023a. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023b. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems*, 30.

Yaqing Wang, Sahaj Agarwal, Subhabrata Mukherjee, Xiaodong Liu, Jing Gao, Ahmed Hassan Awadallah, and Jianfeng Gao. 2022. Adamix: Mixture-of-adaptations for parameter-efficient model tuning. *arXiv preprint arXiv:2210.17451*.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837.

Nan Yang, Tao Ge, Liang Wang, Binxing Jiao, Daxin Jiang, Linjun Yang, Rangan Majumder, and Furu Wei. 2023. Inference with reference: Lossless acceleration of large language models. *arXiv preprint arXiv:2304.04487*.

Qinyuan Ye, Bill Yuchen Lin, and Xiang Ren. 2021. Crossfit: A few-shot learning challenge for cross-task generalization in nlp. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 7163–7189.

Ted Zadouri, Ahmet Üstün, Arash Ahmadian, Beyza Ermiş, Acyr Locatelli, and Sara Hooker. 2023. Pushing mixture of experts to the limit: Extremely parameter efficient moe for instruction tuning. *arXiv preprint arXiv:2309.05444*.

Yun Zhu, Nevan Wichers, Chu-Cheng Lin, Xinyi Wang, Tianlong Chen, Lei Shu, Han Lu, Canoe Liu, Liangchen Luo, Jindong Chen, et al. 2023. Sira: Sparse mixture of low rank adaptation. *arXiv preprint arXiv:2311.09179*.

Aman Madaan, Pranjal Aggarwal, Ankit Anand, Srividya Pranavi Potharaju, Swaroop Mishra, Pei Zhou, Aditya Gupta, Dheeraj Rajagopal, Karthik Kappaganthu, Yiming Yang 等 . 2023. Automix: 自动混合语言模型. arXiv 预印本 arXiv:2310.12963.

Sourab Mangrulkar, Sylvain Gugger, Lysandre Debut, Younes Belkada, Sayak Paul, 和 Benjamin Bossan. 2022. Peft: 最先进的参数高效微调方法. <https://github.com/huggingface/peft>.

Saeed Masoudnia 和 Reza Ebrahimpour. 2014. 专家混合：文献综述。人工智能评论，42:275–293.

Meta. 2023. <https://ai.meta.com/genai/>.

Todor Mihaylov、Peter Clark、Tushar Khot 和 Ashish Sabharwal. 2018 年。《一套盔甲能导电吗？用于开放书籍问答的新数据集》。发表于 2018 年自然语言处理实证方法会议论文集，第 2381–2391 页。

OpenAI. 2023. <https://openai.com/blog/introducing-gpts>.

Ankit Pal、Logesh Kumar Umapathi 和 Malaikannan Sankarasubbu. 2022. Medmcqa: 面向医疗领域问答的大规模多学科多选数据集。在健康、推理与学习会议，第 248–260 页。PMLR。

Guilherme Penedo、Quentin Malartic、Daniel Hesslow、Ruxandra Cojocaru、Alessandro Cappelli、Hamza Alobeidli、Baptiste Pannier、Ebtesam Almazrouei 和 Julien Launay. 2023 年。《用于 Falcon LLM 的精炼网络数据集：用网络数据超越精选语料库，仅使用网络数据》。arXiv 预印本 arXiv:2306.01116。

Edoardo Maria Ponti、Alessandro Sordoni、Yoshua Bengio 和 Siva Reddy. 2023. 结合参数高效模块以实现任务级泛化。载于第 17 届欧洲计算语言学协会分会会议论文集，第 687–702 页。

Or Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A Smith, 和 Mike Lewis. 2022. 测量并缩小语言模型中的组合性差距。arXiv 预印本 arXiv:2210.03350。

Carlos Riquelme, Joan Puigcerver, Basil Mustafa, Maxim Neumann, Rodolphe Jenatton, André Susano Pinto, Daniel Keysers, 和 Neil Houlsby. 2021. 使用稀疏专家混合扩展视觉能力。神经信息处理系统进展，34:8583–8595.

Maarten Sap, Hannah Rashkin, Derek Chen, Ronan Le Bras 和 Yejin Choi. 2019. Social IQA: 关于社会互动的常识推理。发表于 2019 年自然语言处理实证方法会议论文集，第 4463–4473 页。

双子座团队。双子座：一系列高性能多模态模型。技术报告，技术报告，谷歌，2023 年 12 月。网址 <https://storage.googleapis.com>

Steven Tey. 2022. <https://sharegpt.com>.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar 等人 . 2023a. Llama: 开放且高效的基础语言模型。arXiv 预印本 arXiv:2302.13971。

Hugo Touvron、Louis Martin、Kevin Stone、Peter Albert、Amjad Almahairi、Yasmine Babaei、Nikolay Bashlykov、Soumya Batra、Prajwal Bhargava、Shruti Bhosale 等。2023b. Llama 2: 开放基础和微调聊天模型。arXiv 预印本 arXiv:2307.09288。

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, 和 Illia Polosukhin. 2017. 注意力就是你所需要的。神经信息处理系统进展，30.

王雅清、Sahaj Agarwal、Subhabrata Mukherjee、刘晓东、高晶、Ahmed Hassan Awadallah 和高建锋。2022 年。Adamix：用于参数高效模型调优的适应混合。arXiv 预印本 arXiv:2210.17451。

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou 等 . 2022. 连锁思维提示在大型语言模型中引发推理。神经信息处理系统进展，35:24824–24837.

南洋、陶戈、梁旺、焦斌星、姜大新、杨林君、兰根·马朱姆德和魏赴如。2023. 带参考的推理：大语言模型的无损加速。arXiv 预印本 arXiv:2304.04487。

叶勤远，林宇辰，任翔 . 2021. Cross t: 自然语言处理中的跨任务泛化少样本学习挑战 . 载于 2021 年自然语言处理实证方法会议论文集，页码 7163–7189.

Ted Zadouri、Ahmet Üstün、Arash Ahmadian、Beyza Ermiş、Acyr Locatelli 和 Sara Hooker. 2023. 将混合专家推向极限：用于指令调优的极度参数高效 MOE。arXiv 预印本 arXiv:2309.05444。

Yun Zhu, Nevan Wichers, Chu-Cheng Lin, Xinyi Wang, Tianlong Chen, Lei Shu, Han Lu, Canoe Liu, Liangchen Luo, Jindong Chen, 等人 . 2023. Sira: 稀疏低秩适应混合模型 . arXiv 预印本 arXiv:2311.09179.

Crayon: Customized On-Device LLM via Instant Adapter Blending and Edge-Server Hybrid Inference

A Details on Experiments

A.1 Training setting

We implemented the proposed and baseline methods based on the Huggingface PEFT library (Mangrulkar et al., 2022). We set the rank r and scaling factor of a LoRA as 4, and the number of base LoRAs as 32. For training, we use the AdamW optimizer with a learning rate 0.0001 which is cosine annealed. We also set the batch size as 128 and the maximum iteration as 800. For all the methods, we unified the prompt template as shown in Fig. 2 where task cue is not prompted, which is proper to practical use. All the proposed and baseline methods are implemented with PyTorch 2.0.1 and executed on a single NVIDIA A5000 GPU.

A.2 Baselines

Since on-device LLM customization is understudied, we carefully selected three baselines to validate the efficacy of Crayon.

1. **LLaMA** (Touvron et al., 2023a,b) released publicly available models, and also reported the score on the MMLU dataset. However, the reported scores are obtained using the few-shot prompt as in the upper part of Fig. 2, where both the subject name (i.e., the task name) and examples of the subject are given. This few-shot prompt is not applicable to our on-device customization. For fairness, as well as the score from the literature, we also disclose scores using zero-shot prompt of Fig. 2.
2. **Single LoRA** follows the training recipe in the literature (Hu et al., 2021). Since the Single LoRA is trained using the entire training dataset $\mathcal{D}_{tr}$, the same LoRA is used for all the customized tasks. Additionally, for a fair comparison, we fine-tune the Single LoRA (named as "Single LoRA (few-shot)"), which is originally trained on the training dataset, using a few number of examples from customized task $\mathcal{D}_c$. We observed that this additional fine-tuning yields severe performance drop, and it can be attributed to the insufficient size of $\mathcal{D}_c$ for customizing the LoRA to the specific task.

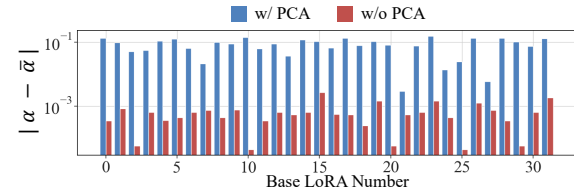


Figure 6: Difference of α and mean of α (i.e., $\bar{\alpha}$) from a data point in training set for each base LoRA whether when using PCA or not.

3. **LoraHub** (Huang et al., 2023) did not focus on tailoring their work for on-device LLM customization, but it can offer a proper baseline for validating Crayon. It outlines how to generate LoRAs specified to a new task by using a given few examples and LoRAs trained for other upstream tasks. However, the downside is the lengthy process required to generate new LoRAs due to the reliance on few-shot learning with the new task's examples. Moreover, due to necessities that all upstream tasks should be clearly defined (i.e., a meticulously refined dataset is needed), it cannot be seamlessly integrated into on-device LLM customization scenarios.

B Details of Tab. 1

Tab. 4 extends the results from Tab. 1 to show the accuracy for each of the 57 subjects in the MMLU dataset. This allows us to see which subjects fall under each category (i.e., STEM, Humanities, Social Science, and Other). Additionally, methodologies with a higher average accuracy also tend to yield higher accuracy across individual subjects.

C Effectiveness of PCA

Fig. 6 illustrates the deviation of the relationship α corresponding to the base LoRAs for one example in the training set, both with and without the use of PCA. The deviation of α when using PCA is larger 100 times than when not using PCA, implying that we can train the base LoRAs more diversely as experts. In line with the one of the objectives of our method, which is to train the base LoRAs with different types of knowledge, we employ PCA to our methodology when getting embeddings.

蜡笔：通过即时适配器混合和边缘服务器混合推理实现的定制化设备端大语言模型

型

关于实验的详情

A.1 培训设置

我们基于 Huggingface PEFT 库实现了所提出的方法和基线方法 (Mangrulkar 等人, 2022)。我们将 LoRA 的秩和缩放因子设置为 4, 基座 LoRA 的数量设置为 32。在训练中, 我们使用 AdamW 优化器, 学习率为 0.0001, 并采用余弦退火。我们还将批量大小设置为 128, 最大迭代次数为 800。对于所有方法, 我们统一了如图 2 所示的提示模板, 其中任务提示未被提示, 这对于实际应用是合适的。所有提出的方法和基线方法都使用 PyTorch 2.0.1 实现, 并在单个 NVIDIA A5000 GPU 上运行。

A.2 基线

由于设备上的大型语言模型定制研究不足, 我们仔细选择了三个基线来验证 Crayon 的有效性。

1. **LLaMA** (Touvron 等人, 2023a,b) 发布了公开可用的模型, 并且也报道了在 MMLU 数据集上的得分。然而, 所报道的得分是使用图 2 上部的少量样本提示获得的, 其中提供了学科名称 (即任务名称) 和该学科的示例。这个少量样本提示不适用于我们的设备端定制。为了公平起见, 以及与文献中的得分进行比较, 我们还使用图 2 上的零样本提示公开了得分。

2. **单 LoRA** 遵循文献中的训练方案 (Hu 等, 2021)。由于单 LoRA 使用整个训练数据集进行训练 $\mathcal{D}_{tr}$, 因此相同的 LoRA 可用于所有定制任务。此外, 为了公平比较, 我们对单 LoRA (命名为“单 LoRA (少量样本)”)进行了微调, 该 LoRA 最初是在训练数据集上训练的, 使用定制任务中的少量示例进行微调 $\mathcal{D}_c$ 。我们观察到, 这种额外微调会造成严重的性能下降, 这可以归因于定制 LoRA 以适应特定任务的数据量不足 $\mathcal{D}_c$ 。

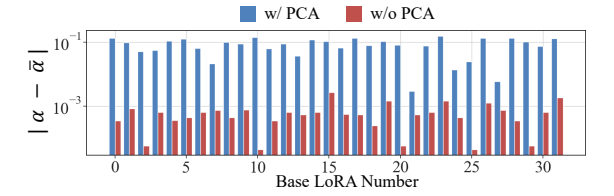


图 6: 每个基础 LoRA 在使用 PCA 或不使用 PCA 时, 从训练集中的数据点计算的 α 与 α 的均值差异 (即 $\bar{\alpha}$)。

3. **LoraHub** (Huang 等, 2023) 并未专注于为设备端大语言模型定制化工作, 但它可以为验证 Crayon 提供一个合适的基准。它概述了如何使用给定的少量示例以及为其他上游任务训练的 LoRAs 来生成针对新任务的 LoRAs。然而, 其缺点是由于依赖于新任务示例的少量学习, 生成新的 LoRAs 过程较为漫长。此外, 由于所有上游任务都必须明确定义 (即需要一个精心优化的数据集), 它无法无缝集成到设备端大语言模型定制场景中。

B 详情表 1

表 4 扩展了表 1 的结果, 以显示 MMLU 数据集中 57 个科目的准确率。这使我们能够看到每个科目落在哪一类 (即 STEM、文科、社会科学和其他)。此外, 平均准确率较高的方法通常在各个科目上的准确率也较高。

C PCA 的有效性

图 6 显示了关系的偏差 α 对应于训练集中一个示例的基础 LoRAs, 在使用和不使用 PCA 的情况下。使用 PCA 时的 α 偏差比不使用 PCA 时大 100 倍, 这意味着我们可以以更多样的方式训练基础 LoRAs 作为专家。符合我们方法的一个目标, 即使用不同类型的知识训练基础 LoRAs, 当获取嵌入时, 我们在方法中采用 PCA。

		LLaMA-7B		LLaMA-13B		Single LoRA		LoRA-Hub	Crayon	Crayon + Hybrid(20%)
		zero-shot	few-shot	zero-shot	few-shot	zero-shot	few-shot			
Abstract Algebra	STEM	28.0	29.0	29.0	34.0	32.0	29.0	32.0	30.0	34.0
Anatomy	Other	32.6	37.0	40.7	45.9	48.9	41.5	52.6	49.6	50.4
Astronomy	STEM	39.5	33.6	44.7	46.1	46.7	42.1	44.7	43.4	50.0
Business Ethics	Other	31.0	40.0	39.0	45.0	33.0	44.0	40.0	36.0	43.0
Clinical Knowledge	Other	34.7	35.1	44.5	45.7	46.4	41.5	43.8	46.0	49.2
College Biology	STEM	30.6	37.5	41.7	45.1	41.7	31.2	46.5	47.9	49.7
College Chemistry	STEM	23.0	32.0	38.0	30.0	31.0	35.0	34.0	33.0	31.0
College Computer Science	STEM	28.0	29.0	36.0	39.0	28.0	16.0	28.0	40.0	39.0
College Mathematics	STEM	30.0	33.0	33.0	32.0	24.0	24.0	30.0	27.0	25.0
College Medicine	Other	30.1	30.6	38.7	42.8	38.7	29.5	34.7	39.9	41.8
College Physics	STEM	17.6	26.5	19.6	18.6	22.5	20.6	24.5	23.5	22.0
Computer Security	STEM	33.0	45.0	56.0	65.0	45.0	39.0	49.0	47.0	55.0
Conceptual Physics	STEM	29.4	36.6	37.4	41.3	38.3	36.6	35.3	34.0	35.1
Econometrics	Social Science	21.1	23.7	30.7	27.2	22.8	29.8	31.6	21.1	23.2
Electrical Engineering	STEM	21.4	26.9	33.8	40.7	35.9	30.3	37.2	42.8	46.2
Elementary Mathematics	STEM	24.3	24.3	27.8	24.9	26.5	25.7	25.7	27.2	28.8
Formal Logic	Humanities	30.2	27.0	36.5	33.3	29.4	30.2	29.4	27.0	30.2
Global Facts	Other	30.0	29.0	30.0	35.0	31.0	33.0	30.0	32.0	32.0
High School Biology	STEM	34.8	34.5	42.9	52.6	49.7	34.8	45.8	51.3	55.9
High School Chemistry	STEM	29.6	28.1	31.0	28.6	34.0	30.5	37.4	41.9	44.6
High School Computer Science	STEM	28.0	31.0	42.0	48.0	31.0	25.0	39.0	41.0	42.0
High School European History	Humanities	33.3	44.2	45.5	61.8	47.9	25.5	52.7	59.4	63.6
High School Geography	Social Science	31.3	34.3	53.0	54.6	56.6	33.8	51.5	60.1	66.2
High School Government And Politics	Social Science	28.5	44.6	62.2	66.3	53.9	36.3	56.5	63.2	67.2
High School Macroeconomics	Social Science	27.2	35.4	38.2	44.4	37.4	30.3	36.9	41.5	43.9
High School Mathematics	STEM	28.9	24.8	27.0	23.7	21.5	25.9	28.9	24.1	23.9
High School Microeconomics	Social Science	25.6	31.9	35.7	47.5	34.9	31.5	37.4	42.4	45.0
High School Physics	STEM	24.5	26.5	30.5	28.5	27.2	23.8	28.5	31.1	31.8
High School Psychology	Social Science	28.3	47.3	52.3	60.9	53.8	36.1	58.5	62.4	65.7
High School Statistics	STEM	22.7	35.2	30.6	30.1	32.9	25.5	33.8	29.2	34.3
High School US History	Humanities	32.8	39.7	45.1	58.3	46.6	28.4	54.9	54.4	60.3
High School World History	Humanities	28.7	40.9	31.6	66.2	51.9	28.7	59.1	59.9	64.8
Human Aging	Other	30.5	40.8	34.1	54.7	41.7	42.2	46.2	42.2	47.5
Human Sexuality	Social Science	30.5	36.6	40.5	58.8	44.3	29.0	50.4	51.9	58.8
International Law	Humanities	42.1	51.2	52.9	62.8	57.0	42.1	58.7	55.4	62.8
Jurisprudence	Humanities	33.3	38.9	50.0	51.9	46.3	37.0	46.3	54.6	56.7
Logical Fallacies	Humanities	29.4	39.3	49.1	52.8	46.6	38.0	51.5	55.8	62.3
Machine Learning	STEM	27.7	23.2	28.6	31.3	29.5	39.3	32.1	34.8	34.8
Management	Other	39.8	35.0	44.7	66.0	56.3	38.8	51.5	58.3	61.5
Marketing	Other	33.8	46.6	64.1	71.8	62.8	50.0	63.2	69.2	72.6
Medical Genetics	Other	36.0	43.0	43.0	52.0	52.0	45.0	48.0	52.0	54.0
Miscellaneous	Other	36.7	42.4	56.4	65.4	60.3	46.0	60.9	62.3	66.5
Moral Disputes	Humanities	29.5	40.2	41.9	50.9	35.0	33.8	42.5	48.8	52.5
Moral Scenarios	Humanities	22.7	24.3	24.6	30.1	24.0	23.9	24.7	24.0	23.5
Nutrition	Other	36.6	37.6	46.7	51.6	45.8	38.9	41.8	50.0	52.2
Philosophy	Humanities	36.0	39.9	45.3	54.0	48.9	35.4	46.6	51.8	57.2
Prehistory	Humanities	39.2	36.1	44.1	51.5	51.9	34.6	50.0	52.8	55.8
Professional Accounting	Other	24.8	25.9	37.6	35.8	29.8	28.7	32.3	34.4	34.0
Professional Law	Humanities	28.1	30.2	34.9	38.0	33.4	27.5	33.9	35.9	36.6
Professional Medicine	Other	30.5	44.5	46.7	50.4	39.0	32.4	37.1	33.8	36.4
Professional Psychology	Social Science	28.3	35.1	41.0	47.7	37.6	32.5	39.5	41.8	46.7
Public Relations	Social Science	35.5	40.9	43.6	60.9	42.7	39.1	48.2	43.6	48.2
Security Studies	Social Science	29.8	31.8	42.9	53.9	31.4	25.7	31.4	38.8	41.5
Sociology	Social Science	35.8	46.8	52.7	61.2	45.3	35.3	54.2	63.2	68.2
US Foreign Policy	Social Science	49.0	46.0	58.0	80.0	60.0	38.0	58.0	68.0	74.0
Virology	Other	34.3	30.1	29.5	43.4	38.0	37.3	34.9	38.6	41.5
World Religions	Humanities	45.6	50.9	63.7	67.8	60.2	45.6	64.3	70.2	70.8
STEM		27.8	34.0	35.0	45.0	33.2	29.7	35.1	36.1	50.3
Humanities		33.1	30.5	43.5	35.8	44.5	33.1	47.3	50.0	50.0
Social Science		30.9	38.3	45.9	53.8	43.4	33.1	46.2	49.8	42.4
Other		33.0	38.1	42.6	53.3	44.6	39.2	44.1	46.0	46.1
Average		31.0	35.1	41.1	46.9	40.7	33.5	42.4	44.6	47.6

Table 4: Detailed results of Tab. 1 on MMLU.

		LLaMA-7B		LLaMA-13B		Single LoRA		LoRA-Hub	Crayon	Crayon + Hybrid(20%)
		zero-shot	few-shot	zero-shot	few-shot	zero-shot	few-shot			
Abstract Algebra	STEM	28.0	29.0	29.0	34.0	32.0	29.0	32.0	30.0	34.0
Anatomy	Other	32.6	37.0	40.7	45.9	48.9	41.5	52.6	49.6	50.4
Astronomy	STEM	39.5	33.6	44.7	46.1	46.7	42.1	44.7	43.4	50.0
Business Ethics	Other	31.0	40.0	39.0	45.0	33.0	44.0	40.0	36.0	43.0
Clinical Knowledge	Other	34.7	35.1	44.5	45.7	46.4	41.5	43.8	46.0	49.2
College Biology	STEM	30.6	37.5	41.7	45.1	41.7	31.2	46.5	47.9	49.7
College Chemistry	STEM	23.0	32.0	38.0	30.0	31.0	35.0	34.0	33.0	31.0
College Computer Science	STEM	28.0	29.0	36.0	39.0	28.0	16.0	28.0	40.0	39.0
College Mathematics	STEM	30.0	33.0	33.0	32.0	24.0	24.0	30.0	27.0	25.0
College Medicine	Other	30.1	30.6	38.7	42.8	38.7	29.5	34.7	39.9	41.8
College Physics	STEM	17.6	26.5	19.6	18.6	22.5	20.6	24.5	23.5	22.0
Computer Security	STEM	33.0	45.0	56.0	65.0	45.0	39.0	49.0	47.0	55.0
Conceptual Physics	STEM	29.4	36.6	37.4	41.3	38.3	36.6	35.3	34.0	35.1
Econometrics	Social Science	21.1	23.7	30.7	27.2	22.8	29.8	31.6	21.1	23.2
Electrical Engineering	STEM	21.4	26.9	33.8	40.7	35.9	30.3	37.2	42.8	46.2
Elementary Mathematics	STEM	24.3	24.3	27.8	24.9	26.5	25.7	25.7	27.2	28.8
Formal Logic	Humanities	30.2	27.0	36.5	33.3	29.4	30.2	29.4	27.0	30.2
Global Facts	Other	30.0	29.0	30.0	35.0	31.0	33.0	30.0	32.0	32.0
High School Biology	STEM	34.8	34.5	42.9	52.6	49.7	34.8	45.8	51.3	55.9
High School Chemistry	STEM	29.6	28.1	31.0	28.6	34.0	30.5	37.4	41.9	44.6
High School Computer Science	STEM	28.0	31.0	42.0	48.0	31.0	25.0	39.0	41.0	42.0
High School European History	Humanities	33.3	44.2	45.5	61.8	47.9	25.5	52.7	59.4	63.6
High School Geography	Social Science	31.3	34.3	53.0	54.6	56.6	33.8	51.5	60.1	66.2
High School Government And Politics	Social Science	28.5	44.6	62.2	66.3	53.9	36.3	56.5	63.2	67.2
High School Macroeconomics	Social Science	27.2	35.4	38.2	44.4	37.4	30.3	36.9	41.5	43.9
High School Mathematics	STEM	28.9	24.8	27.0	23.7	21.5	25.9	28.9	24.1	23.9
High School Microeconomics	Social Science	25.6	31.9	35.7	47.5	34.9	31.5	37.4	42.4	45.0
High School Physics	STEM	24.5	26.5	30.5	28.5	27.2	23.8	28.5	31.1	31.8
High School Psychology	Social Science	28.3	47.3	52.3	60.9	53.8	36.1	58.5	62.4	65.7
High School Statistics	STEM	22.7	35.2	30.6	30.1	32.9	25.5	33.8	29.2	34.3
High School US History	Humanities	32.8	39.7	45.1	58.3	46.6	28.4	54.9	54.4	60.3
High School World History	Humanities	28.7	40.9	31.6	66.2	51.9	28.7	59.1	59.9	64.8
Human Aging	Other	30.5	40.8	34.1	54.7	41.7	42.2	46.2	42.2	47.5
Human Sexuality	Social Science	30.5	36.6	40.5	58.8	44.3	29.0	50.4	51.9	58.8
International Law	Humanities	42.1	51.2	52.9	62.8	57.0	42.1	58.7	55.4	62.8
Jurisprudence	Humanities	33.3	38.9	50.0	51.9	46.3	37.0	46.3	54.6	56.7
Logical Fallacies	Humanities	29.4	39.3	49.1	52.8	46.6	38.0	51.5	55.8	62.3
Machine Learning	STEM	27.7	23.2	28.6	31.3	29.5	39.3	32.1	34.8	34.8
Management	Other	39.8	35.0	44.7	66.0	56.3	38.8	51.5	58.3	61.5
Marketing	Other	33.8	46.6	64.1	71.8	62.8	50.0	63.2	69.2	72.6
Medical Genetics	Other	36.0	43.0	43.0	52.0	52.0	45.0	48.0	52.0	54.0
Miscellaneous	Other	36.7	42.4	56.4	65.4	60.3	46.0	60.9	62.3	66.5
Moral Disputes	Humanities	29.5	40.2	41.9	50.9	35.0	33.8	42.5	48.8	52.5
Moral Scenarios	Humanities	22.7	24.3	24.6	30.1	24.0	23.9	24.7	24.0	23.5
Nutrition	Other	36.6	37.6	46.7	51.6	45.8	38.9	41.8	50.0	52.2
Philosophy	Humanities	36.0	39.9	45.3	54.0	48.9	35.4	46.6	51.8	57.2
Prehistory	Humanities	39.2	36.1	44.1	51.5	51.9	34.6	50.0	52.8	55.8
Professional Accounting	Other	24.8	25.9	37.6	35.8	29.8	28.7	32.3	34.4	34.0
Professional Law	Humanities	28.1	30.2	34.9	38.0	33.4	27.5	33.9	35.9	36.6
Professional Medicine	Other	30.5	44.5	46.7	50.4	39.0	32.4	37.1	33.8	36.4
Professional Psychology	Social Science	28.3	35.1	41.0	47.7	37.6	32.5	39.5	41.8	46.7
Public Relations	Social Science	35.5	40.9	43.6	60.9	42.7	39.1	48.2	43.6	48.2
Security Studies	Social Science	29.8	31.8	42.9	53.9	31.4	25.7	31.4	38.8	41.5
Sociology	Social Science	35.8	46.8	52.7	61.2	45.3	35.3	54.2	63.2	68.2
US Foreign Policy	Social Science	49.0	46.0	58.0	80.0	60.0	38.0	58.0	68.0	74.0
Virology	Other	34.3	30.1	29.5	43.4	38.0	37.3	34.9	38.6	41.5
World Religions	Humanities	45.6	50.9	63.7	67.8	60.2	45.6	64.3	70.2	70.8
STEM		27.8	34.0	35.0	45.0	33.2	29.7	35.1	36.1	50.3
Humanities		33.1	30.5	43.5	35.8	44.5	33.1	47.3	50.0	50.0
Social Science		30.9	38.3	45.9	53.8	43.4	33.1	46.2	49.8	42.4
Other		33.0	38.1	42.6	53.3	44.6	39.2	44.1	46.0	46.1
Average		31.0	35.1	41.1	46.9	40.7	33.5	42.4	44.6	47.6